

ARCHITECTURE LOGICIELLE: PRINCIPES

I. Définition et Contexte

I.1 Archi Business

I.2 Archi Fonctionnelle

I.3 Archi Applicative

I.4 Archi Technique

I.5 Archi logicielle

II. Principes d'Architecture

II.1 Pas de Liberté sans Contraintes

II.2 Principes S.O.L.I.D

II.3 Cohésion des composants

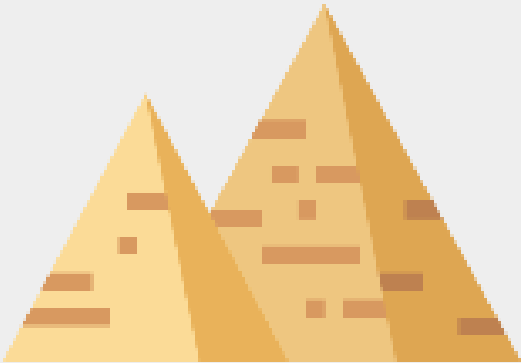
II.4 Couplage des composants

II.5 Synthèse: Architecte Propre

I. Définition et Contexte

étymologie

- ἀρχιτέκτων, **arkhitektôn** « maitre-charpentier »
 - *chef, directeur* => monarque
- ἀρχαιολόγος, **arkhaiologos** « discours sur le début »
- ἀρχή, **arkhé** => *principe directif / structurant / fondateur*

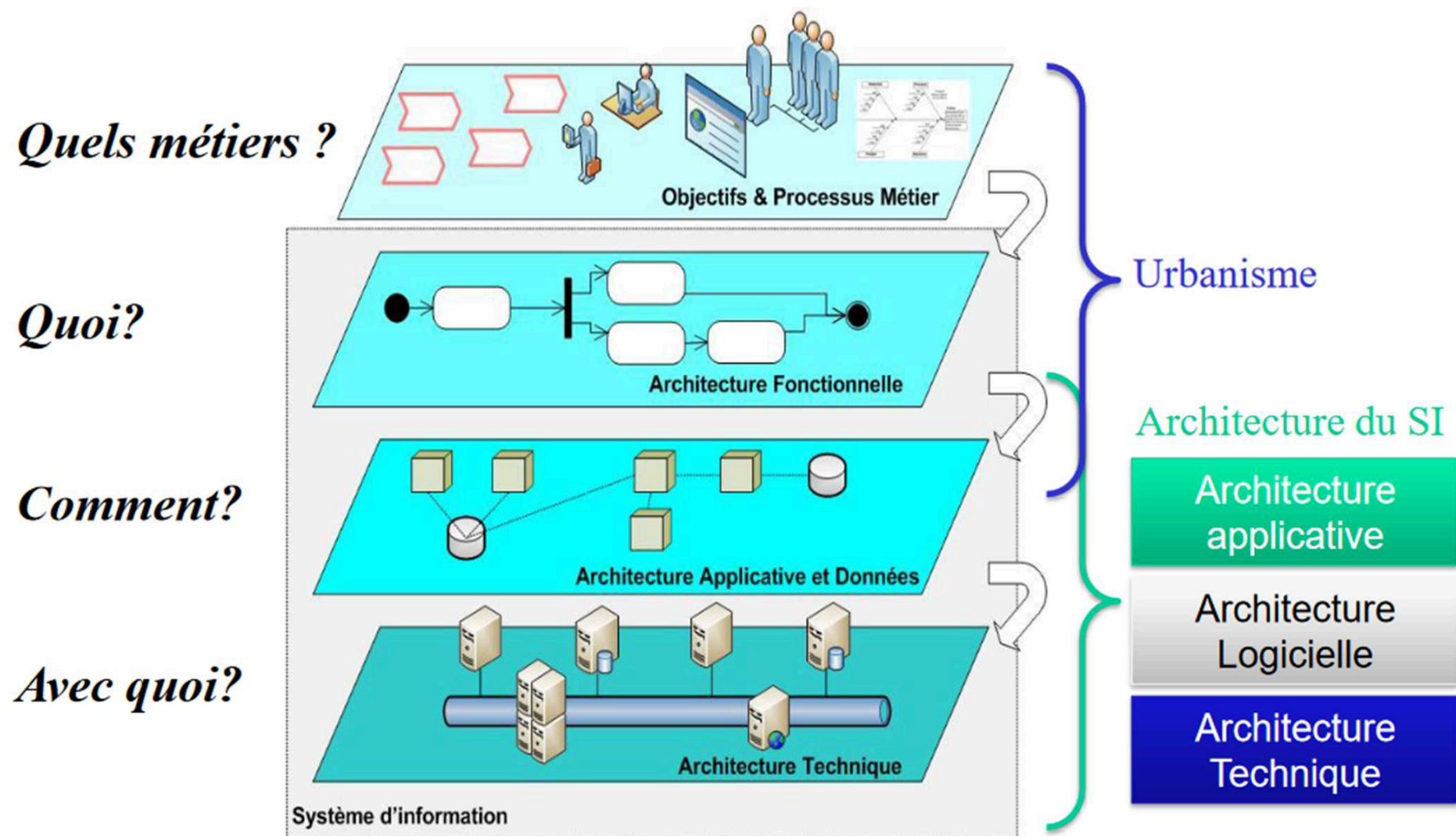


archaïque ou bien structuré ?

Cadres d'architecture d'entreprise

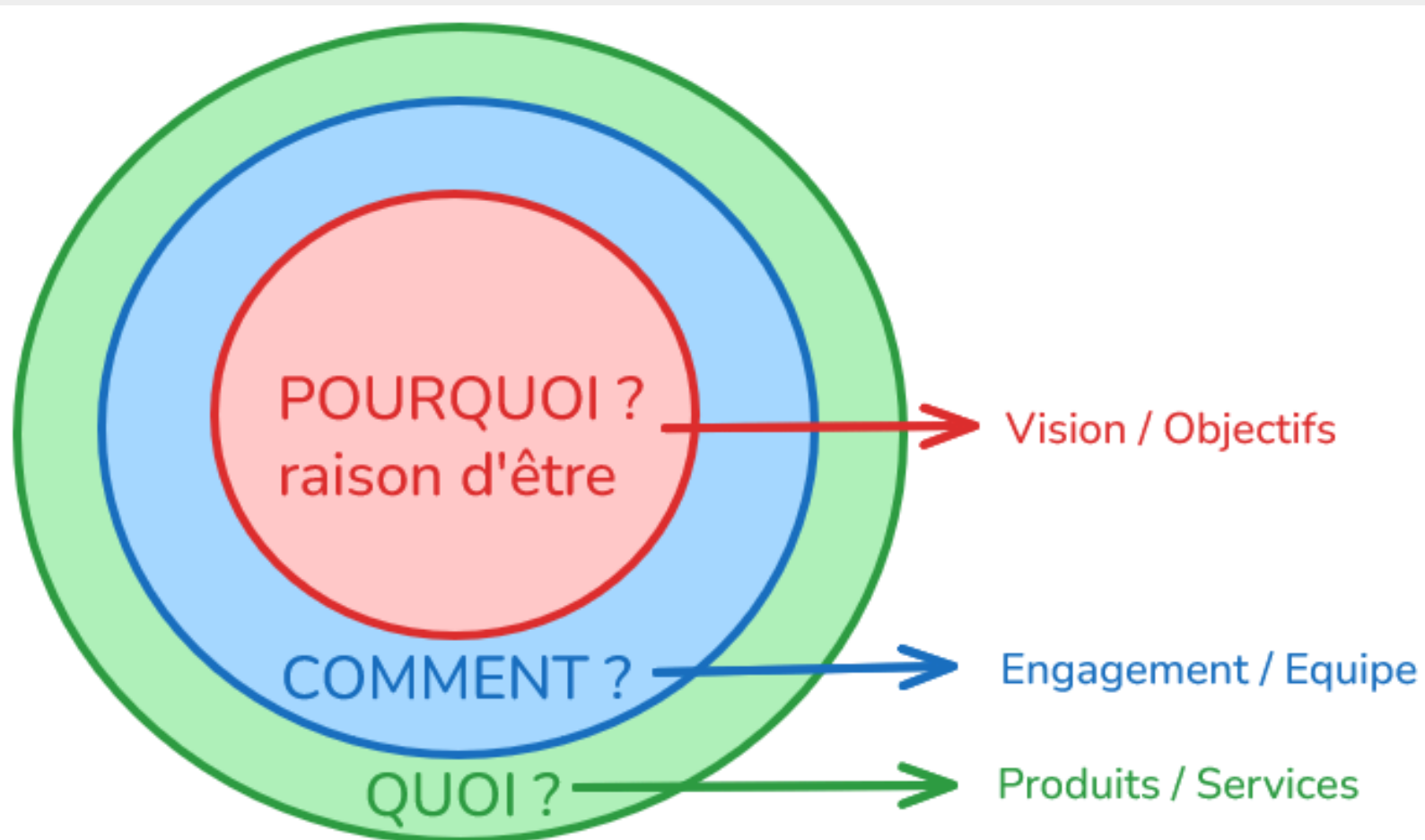
- T.O.G.A.F: **T**he **O**pen **G**roup **A**rchitecture **F**ramework (1995)
- Zachman
- G.E.A.F: **G**artner **E**nterprise **A**rchitecture **F**ramework
- DoDAF / NAF: **D**epartment of **D**efense (US) / **N**ATO (OTAN)
- Carnegie Mellon University: **S**oftware **E**ngineering **I**nstitute (SEI)
=> *C4 model* de Simon Brown

Strates de d'architecture selon TOGAF



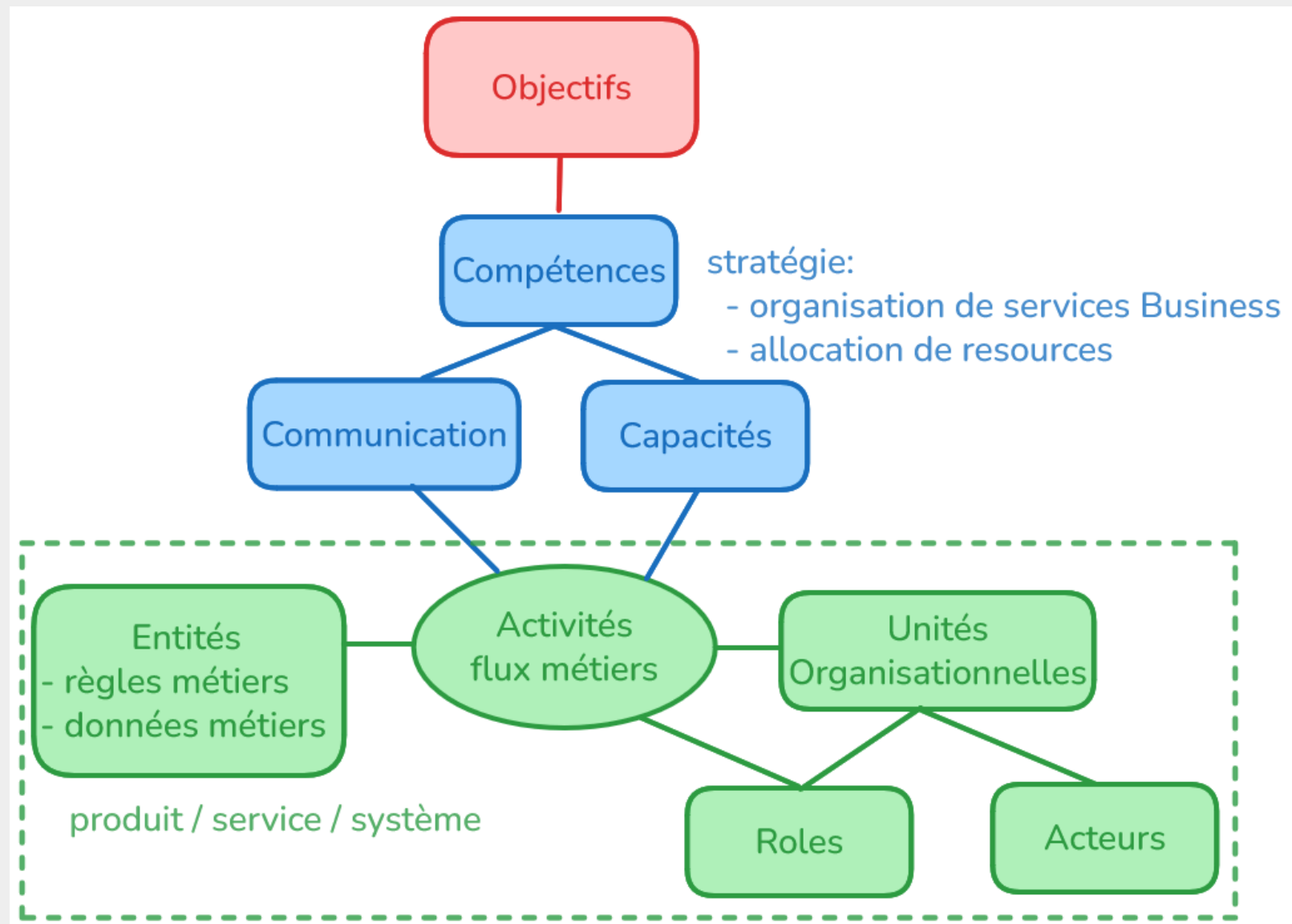
I.1 Archi Business

- golden circle - Simon Sinek

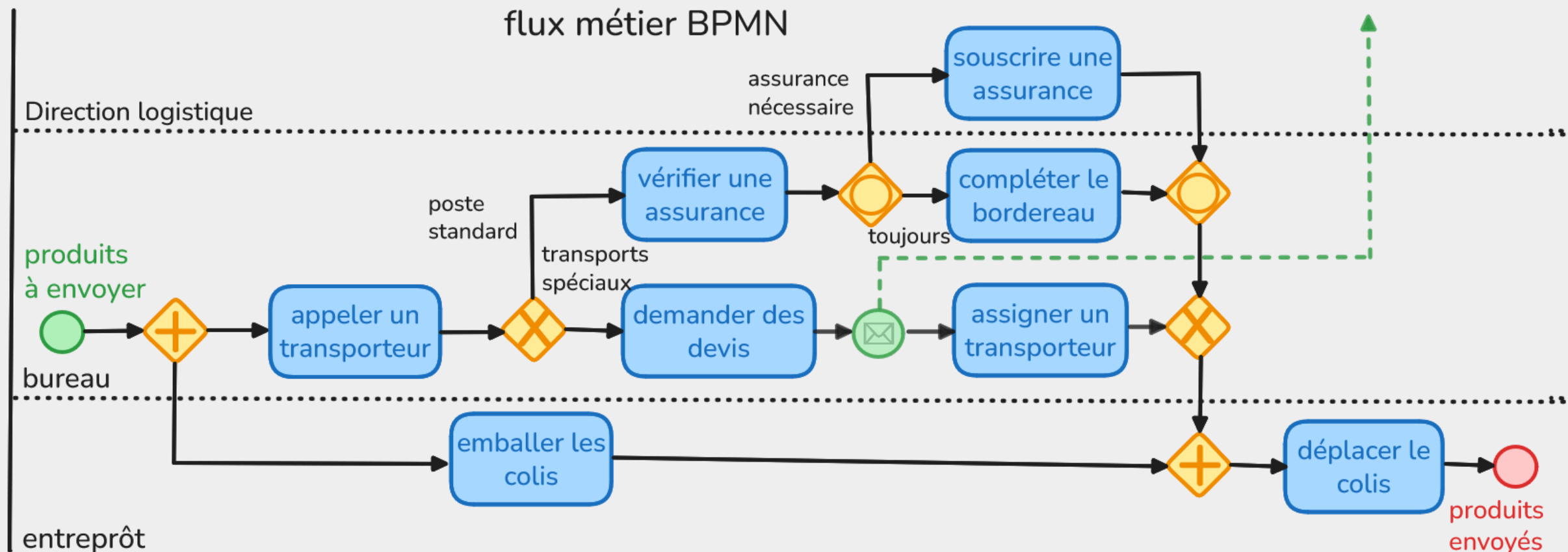


Archi Business

- méta-modèle



Archi Business: flux métiers

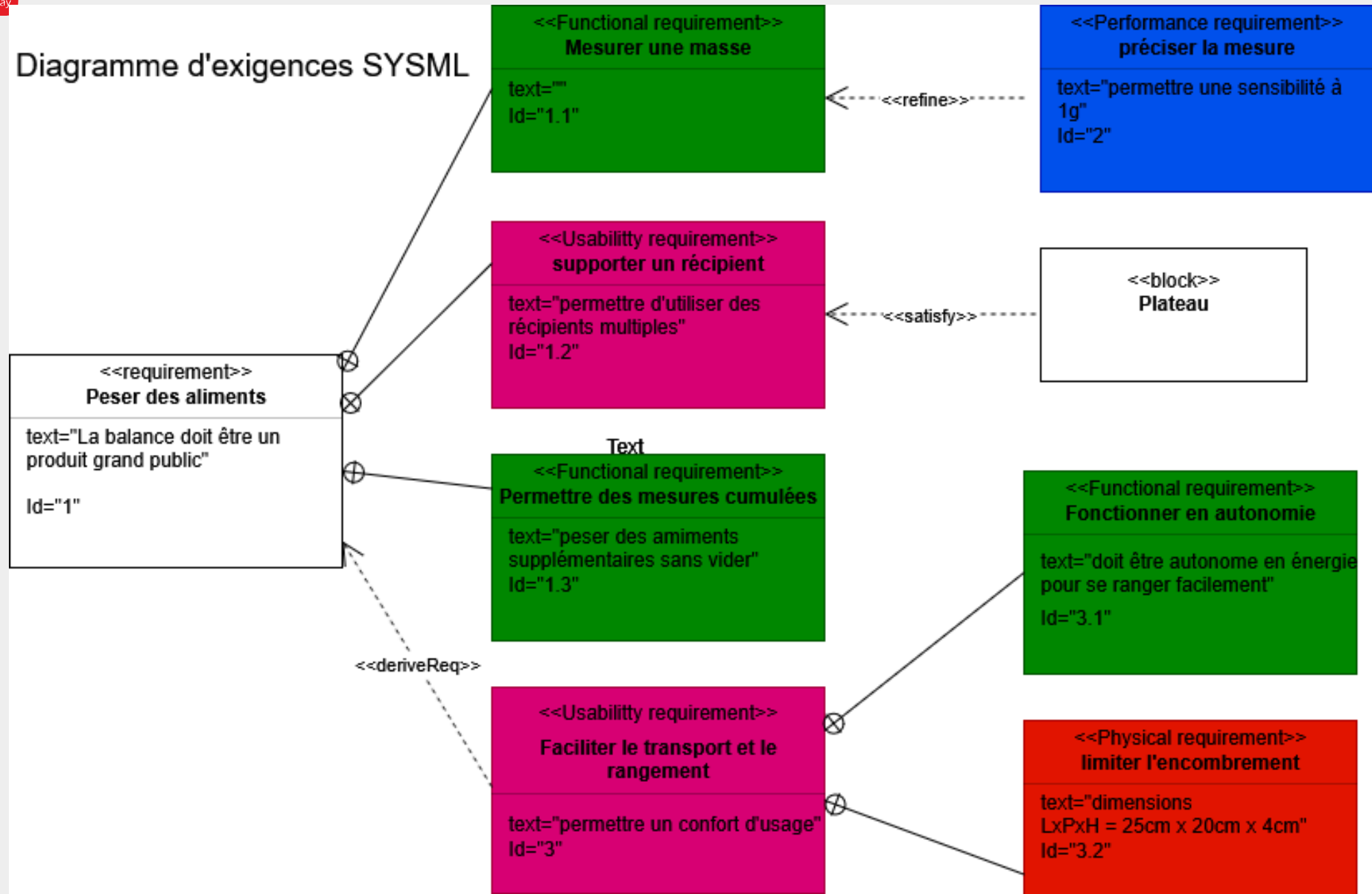


- B.P.M.N: **B**usiness **P**rocess **M**odel and **N**otation

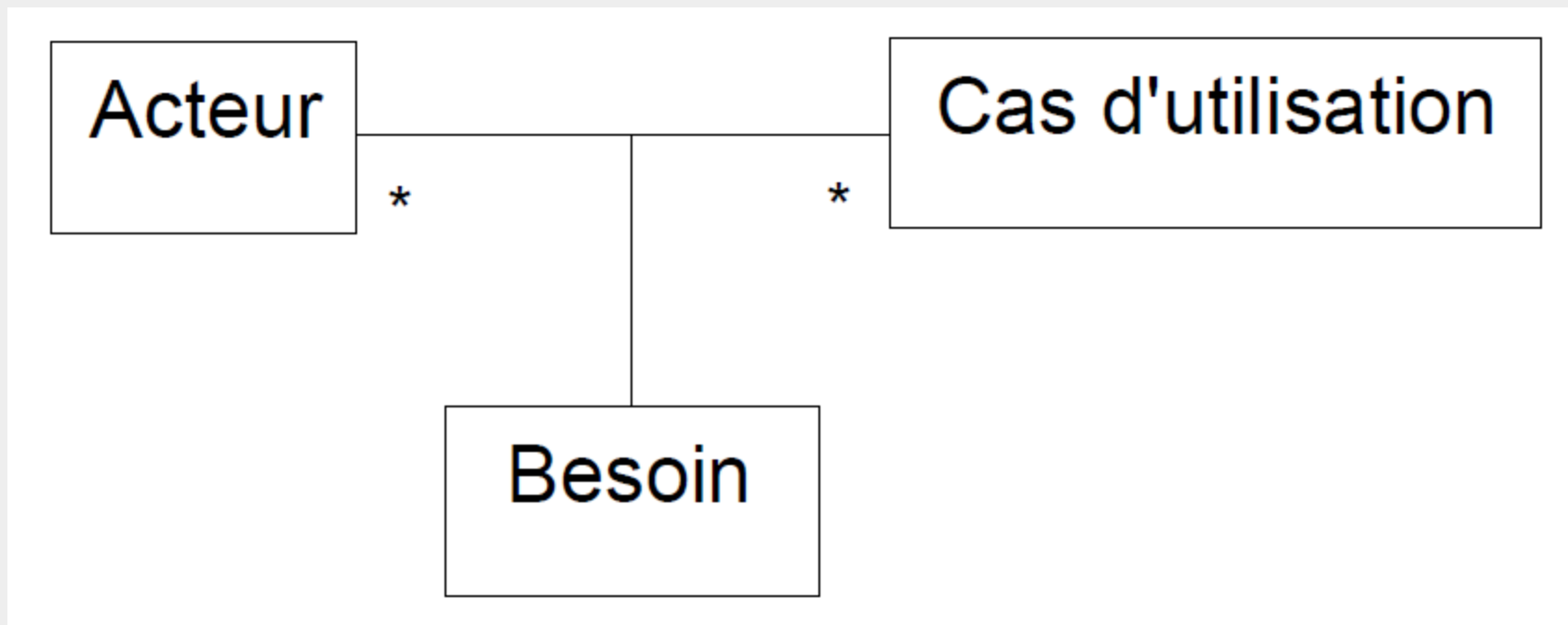
I.2 Archi Fonctionnelle

- **passage à l'abstraction** des flux métiers
1. analyse **fonctionnelle** et des **exigences**
 2. fixation des **cas d'usage** d'*Ivar Jacobson*
 - lien entre les **acteurs** et les features ou entre features
 3. fixation des **entités** comme objets métiers globaux d'entreprise
 - *règles et données* métier **critiques** => Business
- “ **Pas de détail !** pas de technos / framework / protocole / infra
il faut décider les détails au plus tard !! ”

Diagramme d'exigences SYSML

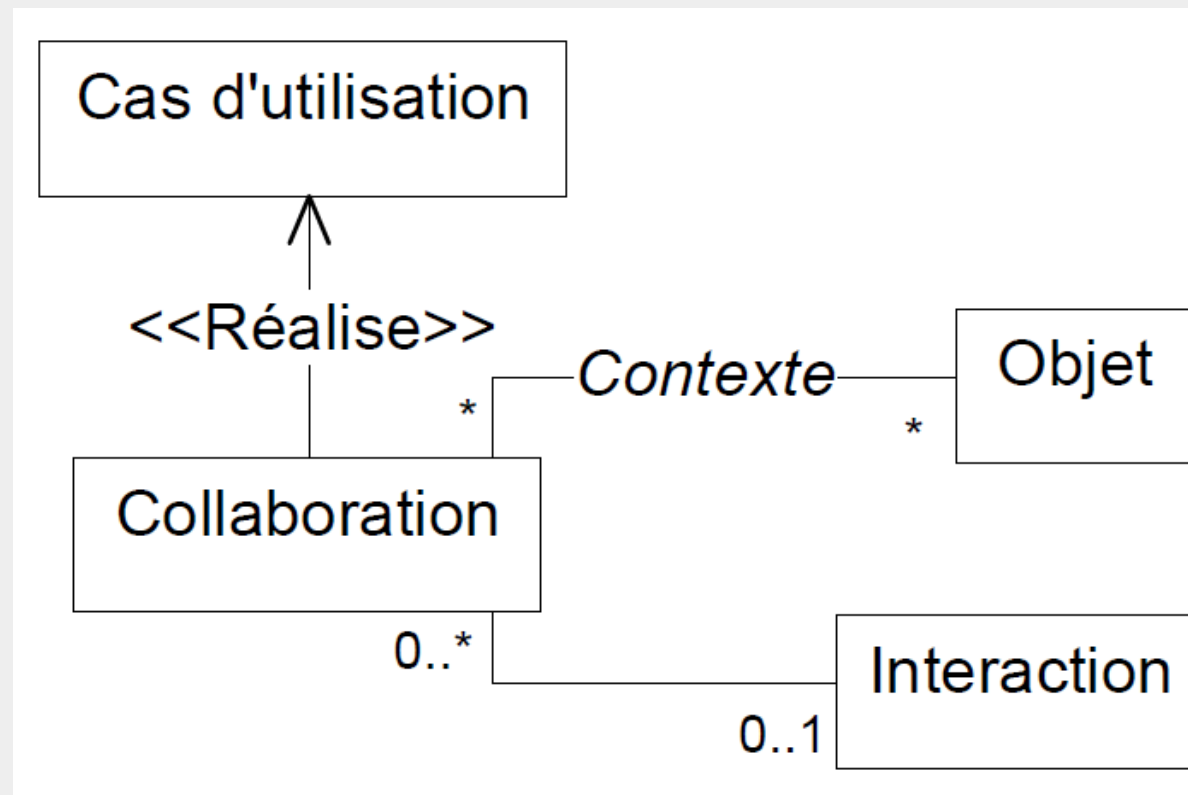


Archi Fonct°: cas d'usage



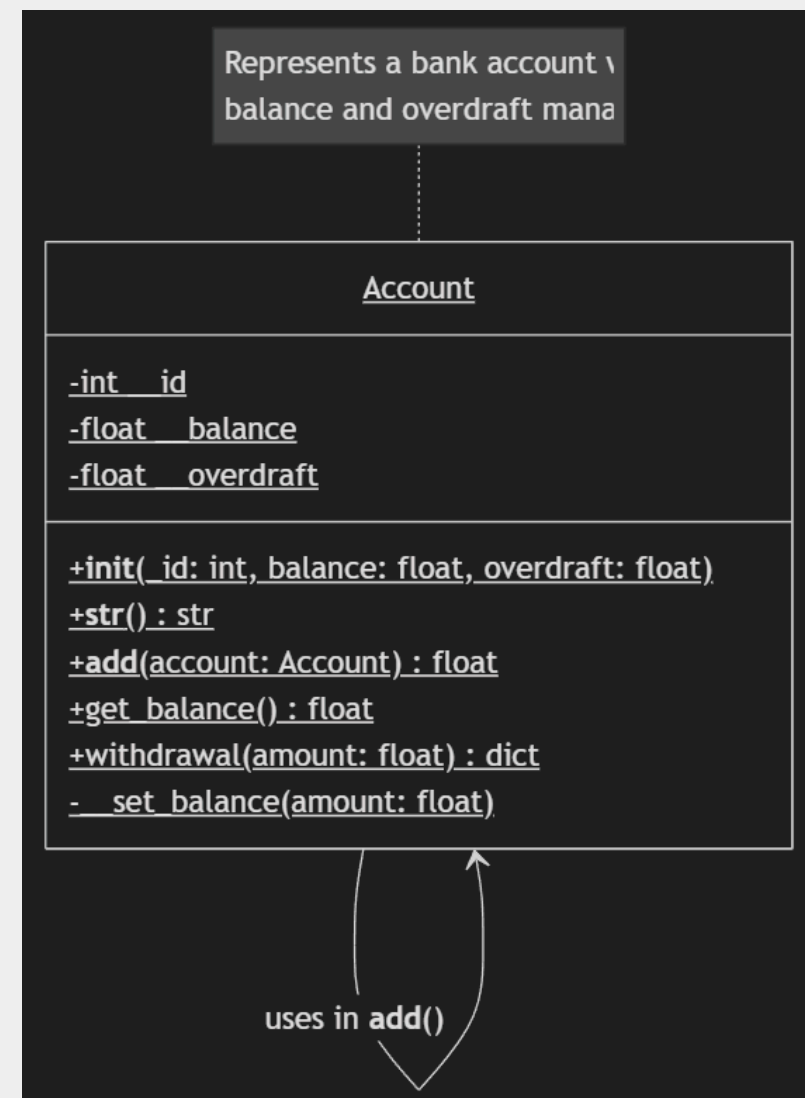
Archi Fonct°: collaboration

- pont entre *cas d'usage* et *entités*
- interaction: message entre
 - acteur et entité
 - ou entre entités



Archi Fonct°: classe / entité

- lister tous les attributs et méthodes peut être contre productif
 - certaines méthodes privées
 - les getter / setters
- “ ici une représentation **non exhaustive serait plus exploitable !** ”

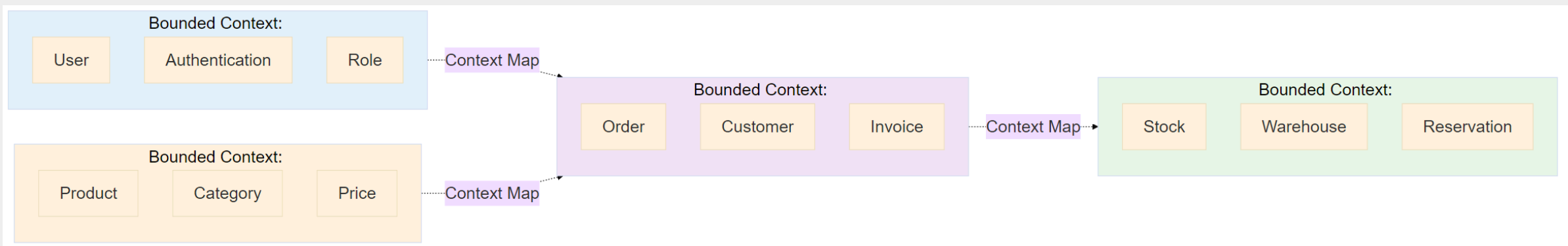


Archi Fonct°: lisibilité

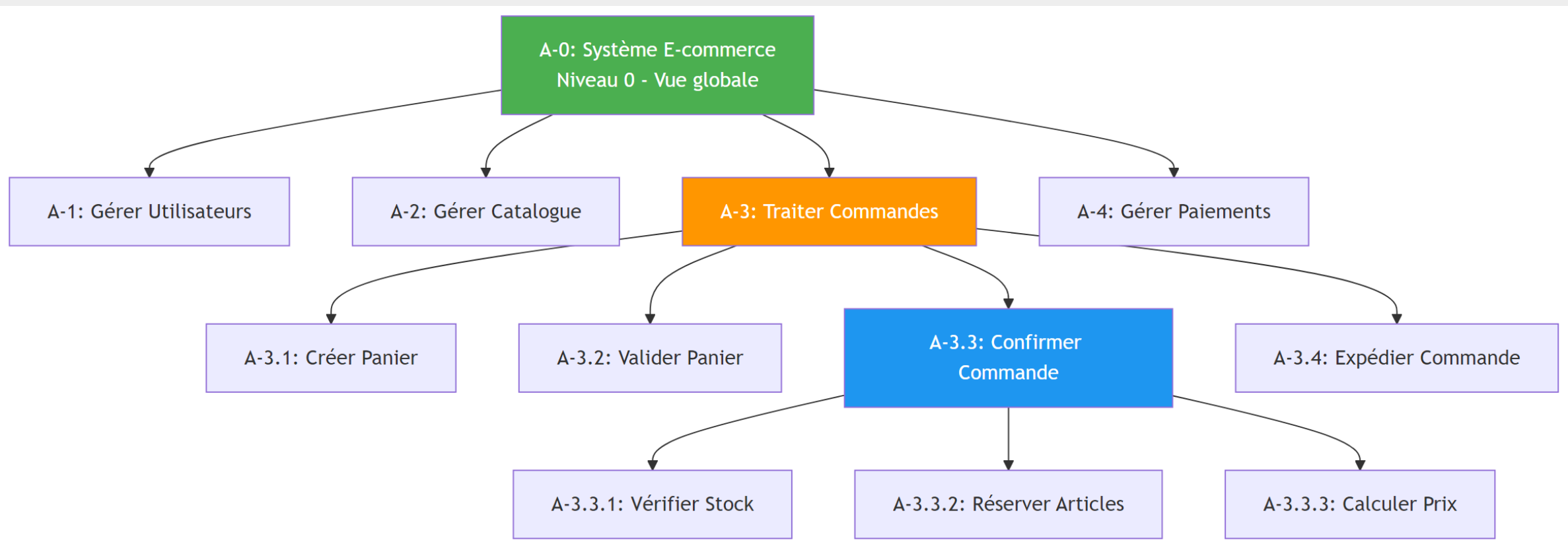
“ *ne pas éditer une vue architecturale d'un bloc* ”

- découpe par *domaines métier*:
 - "Bounded Context"
- décomposition *hiérarchique*
 - "**S**tructured **A**nalysis and **D**esign **T**echnique"
- découpe par *Acteur*

Ex: découpe de classes par contextes (thématiques)



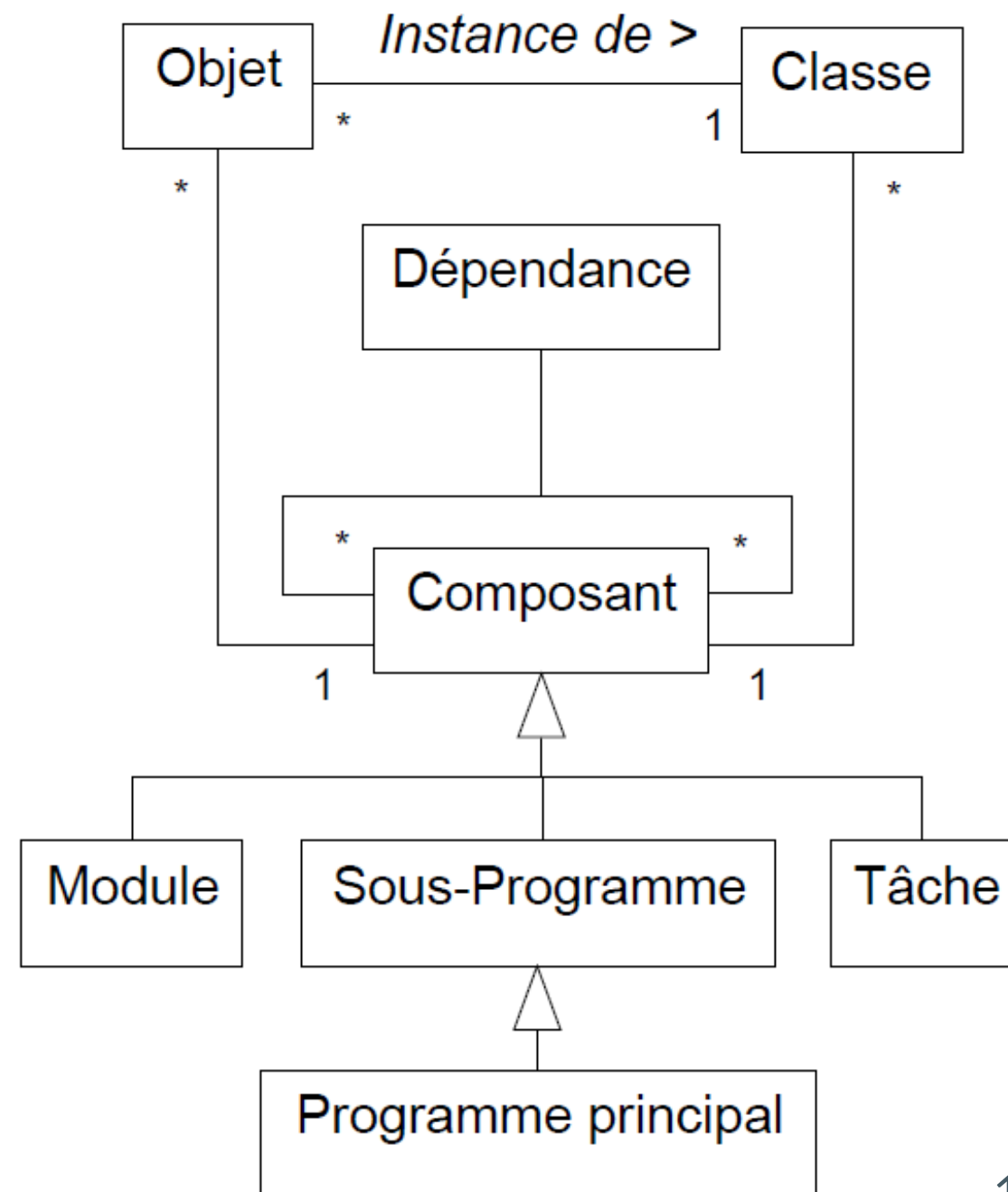
Ex: découpe hiérarchique d'exigences (SADT)



I.3 Archi Applicative

- **Structure** qui définit comment les *applications*
 - *réalisent* l'architecture fonctionnelle
 - gèrent les **données** de l'organisation.
 - **sont alignés** avec les besoins métiers
- notion centrale: **composant logiciel**
 - *unité de déploiement autonome*
 - composé d'un ou plusieurs **modules** logiciels
 - vus comme *assemblage cohérent* de classes : **sans détail**

Archi Applicative: composants

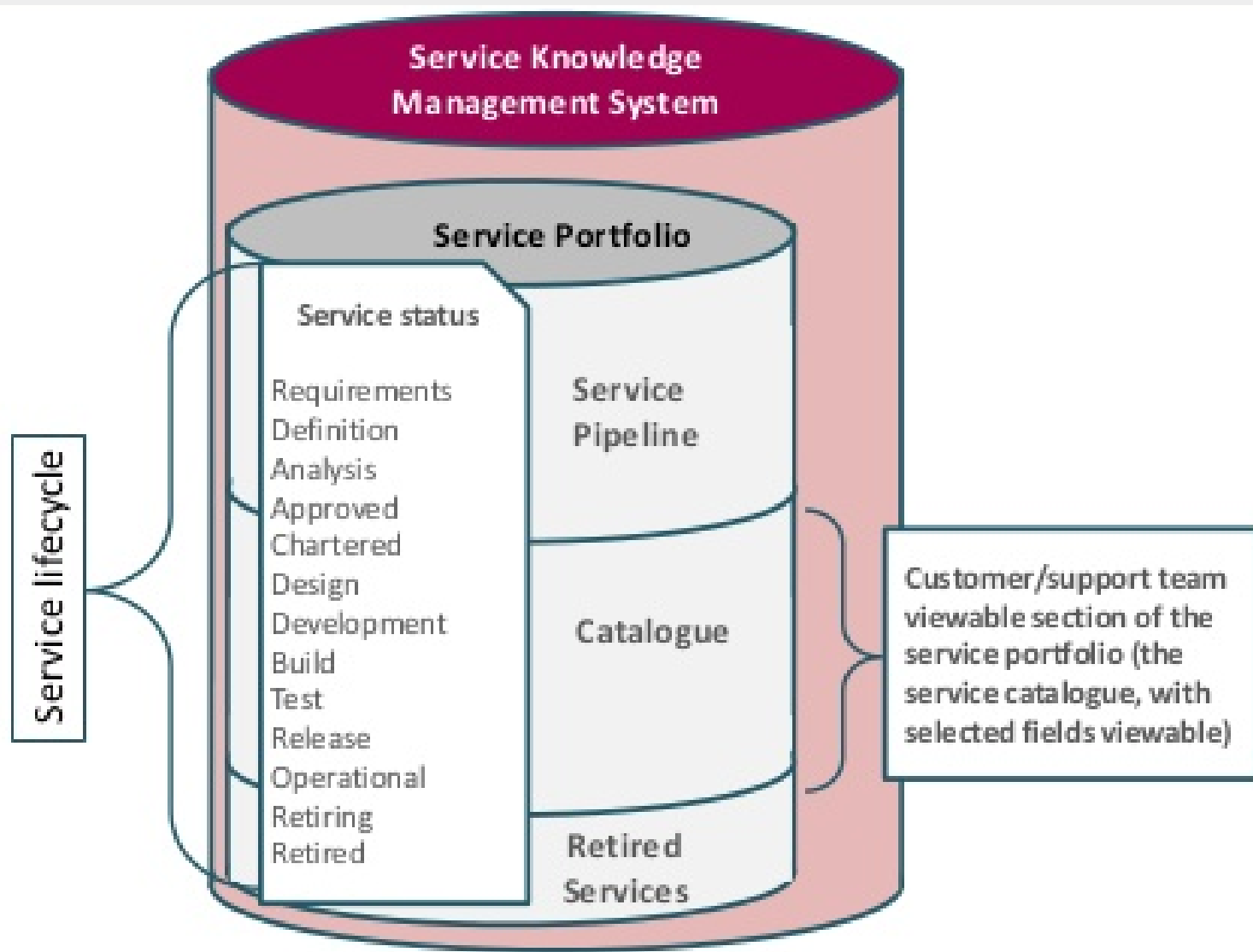


Archi applicative: détails

composants techniques clés

- *CMDB*: Référentiel central et unique des configurations
- *Portfolio de services*: Cycle de vie
- Interactions entre applications
 - *protocoles*: HTTP, SSH, TCP ...
 - *langages d'API*: SOAP, REST, GraphQL
 - *formats de données / de journaux*: Json, YAML
- *technologies*: frameworks (Symfony) / BDD (Mysql) / UI (Nginx)

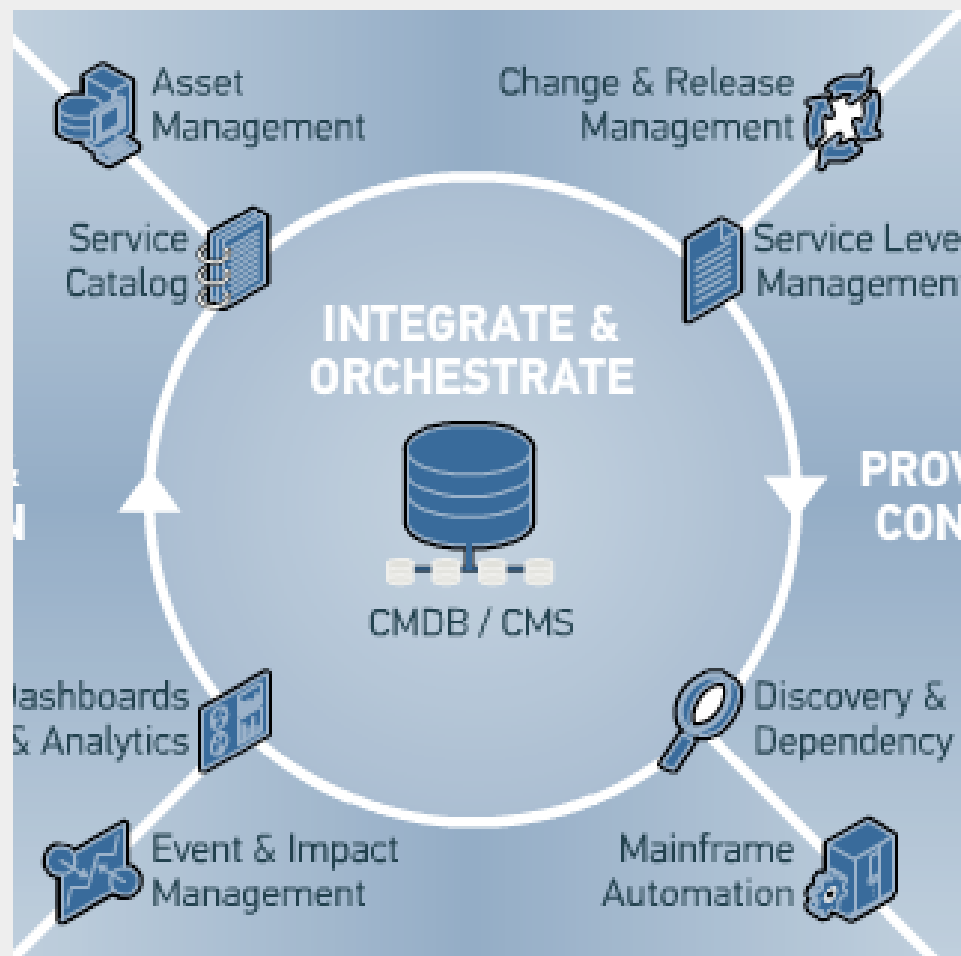
Archi applicative: portfolio



Archi applicative: Configuration Management DB

- capacités:
 - inventaire des machines ex: *OCS*
 - référentiel contacts: ex: *LDAP*
- contraintes: SLAs - niveaux de service
- activités: portfolio services
- utilisées par des processus
 - exploration / CI / CD / observabilité

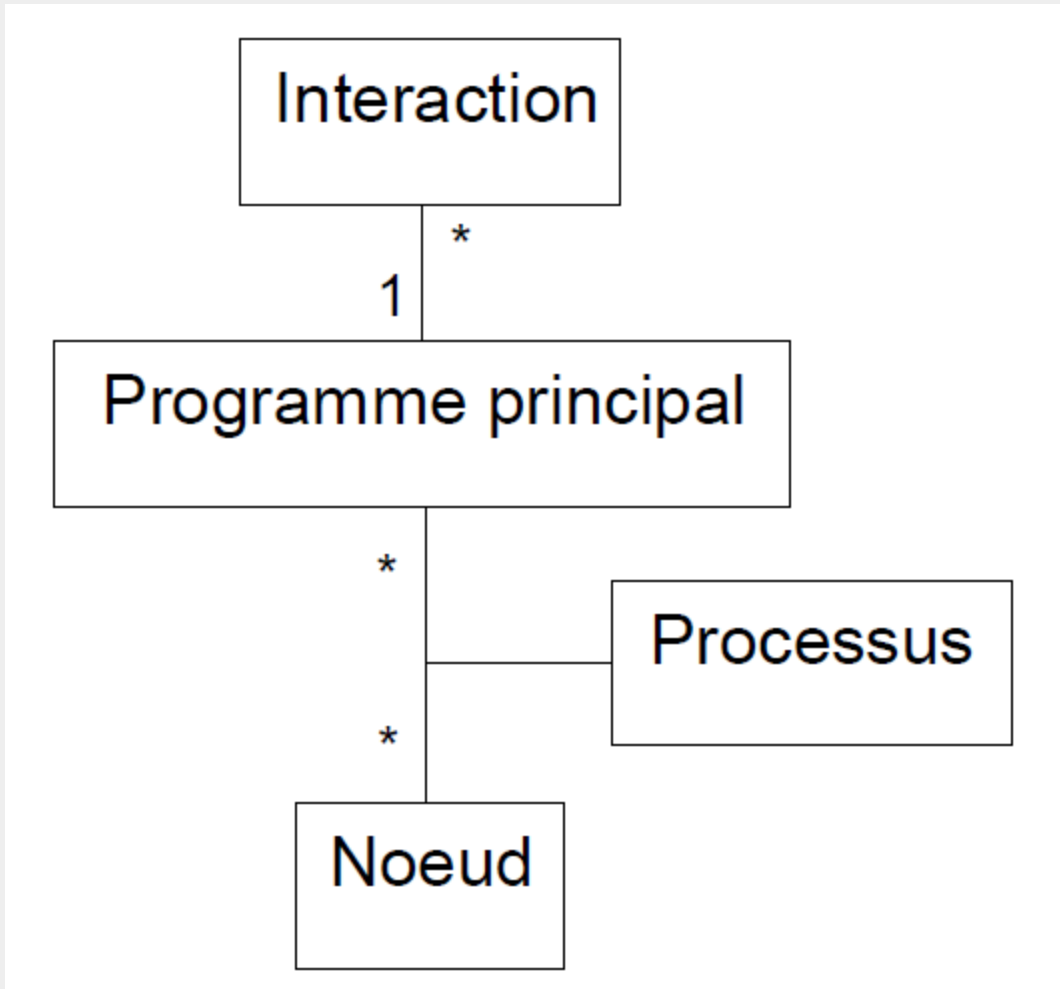
“ *source unique de vérité de la DSI !* ”



I.4 Archi Technique

- **Structure** qui définit l'assemblage des
 - composants matériels
 - composants techniques
 - dont réseaux
- nécessaires au **déploiement** et au fonctionnement des applications

Archi Technique: Déploiement



Archi Technique: détails - infra materielle

- Serveurs
- Stockage: *SAN, NAS, object storage*
- Réseaux: *LAN, WAN, SDN*
- Équipements: *load balancers, firewalls*

Archi Technique: détails - services d'infra

- Virtualisation: *VMware, Hyper-V*
- Conteneurisation: *Docker, Kubernetes*
- middlewares *MoM, ESB, API gateway ...*
- Cloud IaaS / PaaS / SaaS *AWS, GCP, Azure, Nextcloud ...*
- Supervision: *Prometheus, Zabbix ...*

Archi Technique: détails Sécurité technique

- Authentification et autorisation: *SSO ...*
- Chiffrement *Vault, Kerberos ...*
- Pare-feu et DMZ
- **S**ecurity **I**nformation & **E**vent **M**anagement *Wazuh ...*

I.5 Archi logicielle

- logiciel « *software* »: matière **molle / malléable**
- qui peut changer *facilement* et change *souvent*

définition normative:

“ **Architecture Logicielle:**
Description symbolique et schématique des différents
composants d'un système informatique, leur *relations et interactions*. ”

=> *Partager* un même niveau de connaissance

=> Prendre en compte *l'état de l'art*

Archi logicielle: principal objectif

Faciliter l'intégration des **changements de structure**

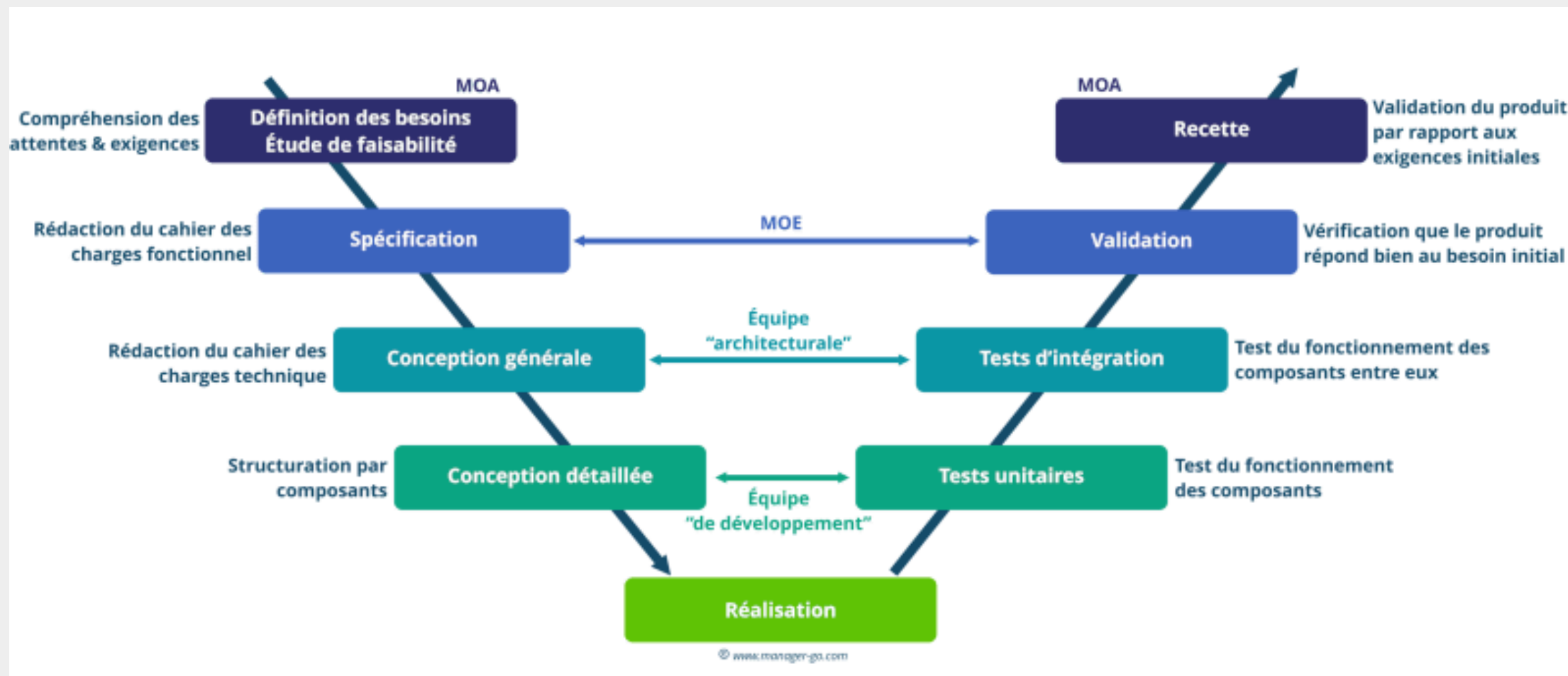
- Éviter la *redondance* et les silos
 - garantir l'*interopérabilité* des composants
 - Optimiser les *coûts et la maintenance*
 - Supporter l'*évolution* et la *mise en échelle*
- “ la structure logicielle est déterminée **sans détails**
la structure logicielle rassemble surtout des éléments
d'*architecture fonctionnelle et applicative* ”

Archi logicielle: assertions

- Architecture != Conception **Détaillée**
- Architecture != *infrastructure*
- Architecture != *technologie* fétiche de l'architecte
- Architecture != oeuvre *d'un seul* architecte
- Architecture != 1 schéma
- l'Architecture demande
 - des données
 - des métriques qualités
 - critères d'acceptation

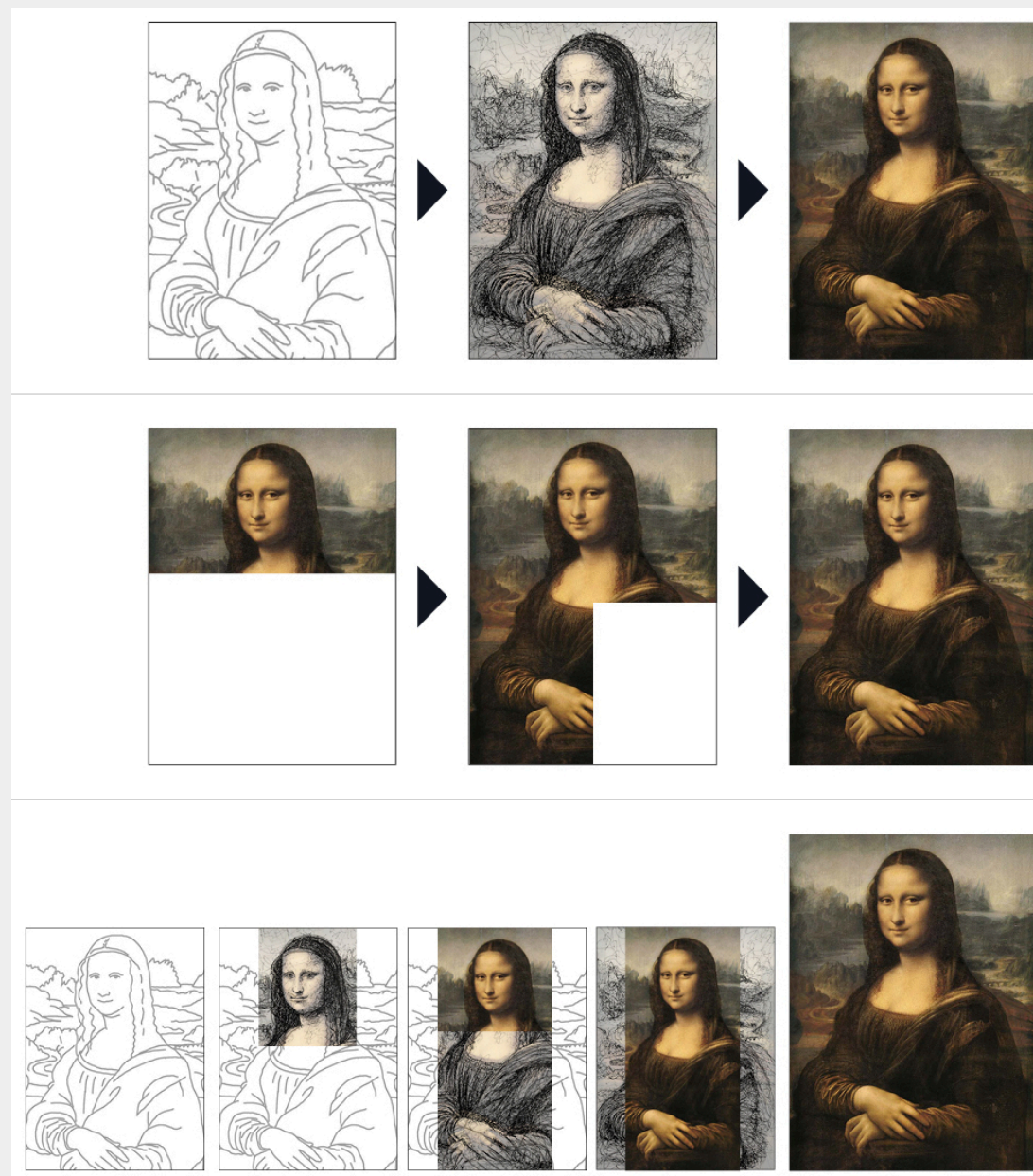
Archi logicielle: quand la positionner dans le projet

- Si **prédictif** => 1 cycle en V | si **agile** => archi sur sprint0 + MAJs

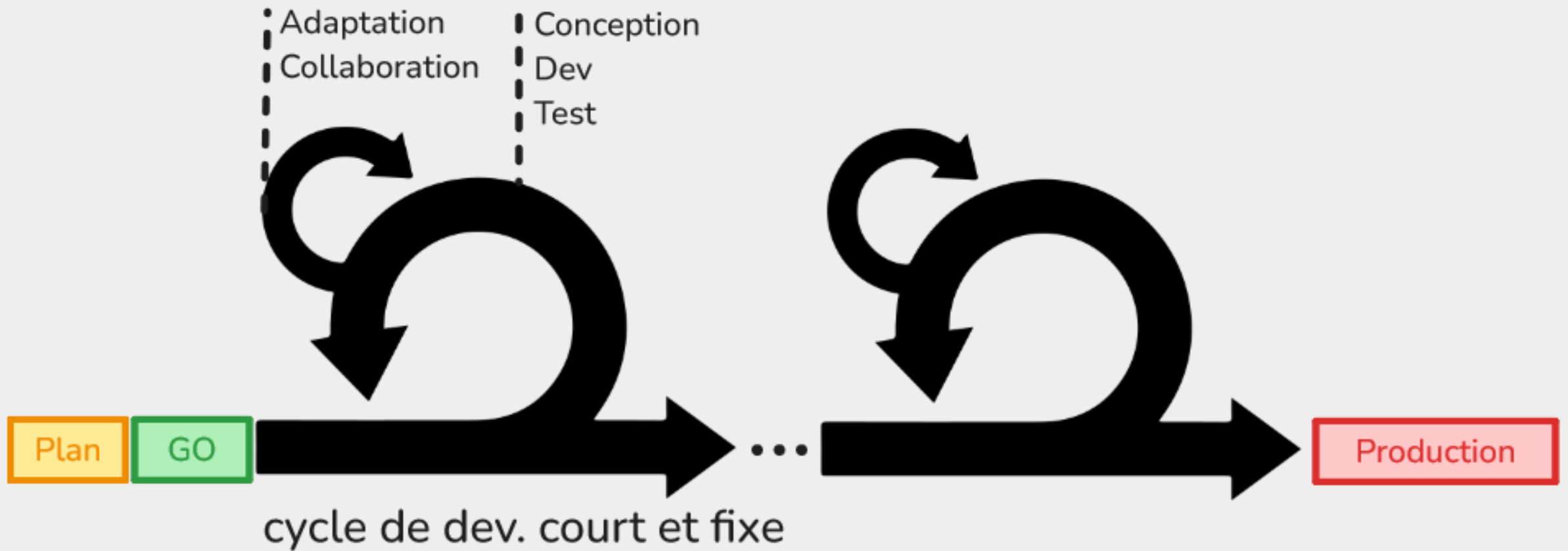


Cycle Agile

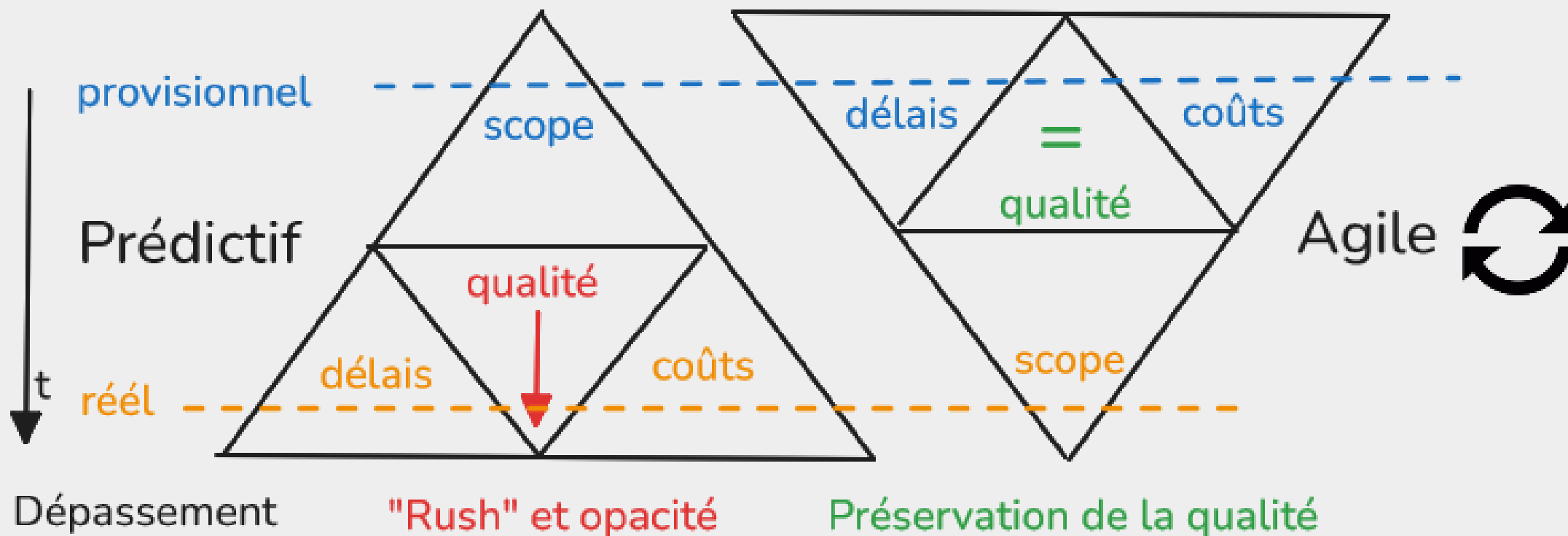
- **cycle itératif**
=> montée en qualité / versions
- **cycle incrémental**
=> le tout fait de ses parties
- **cycle itératif ET incrémental**
=> montée en version d'un incrément



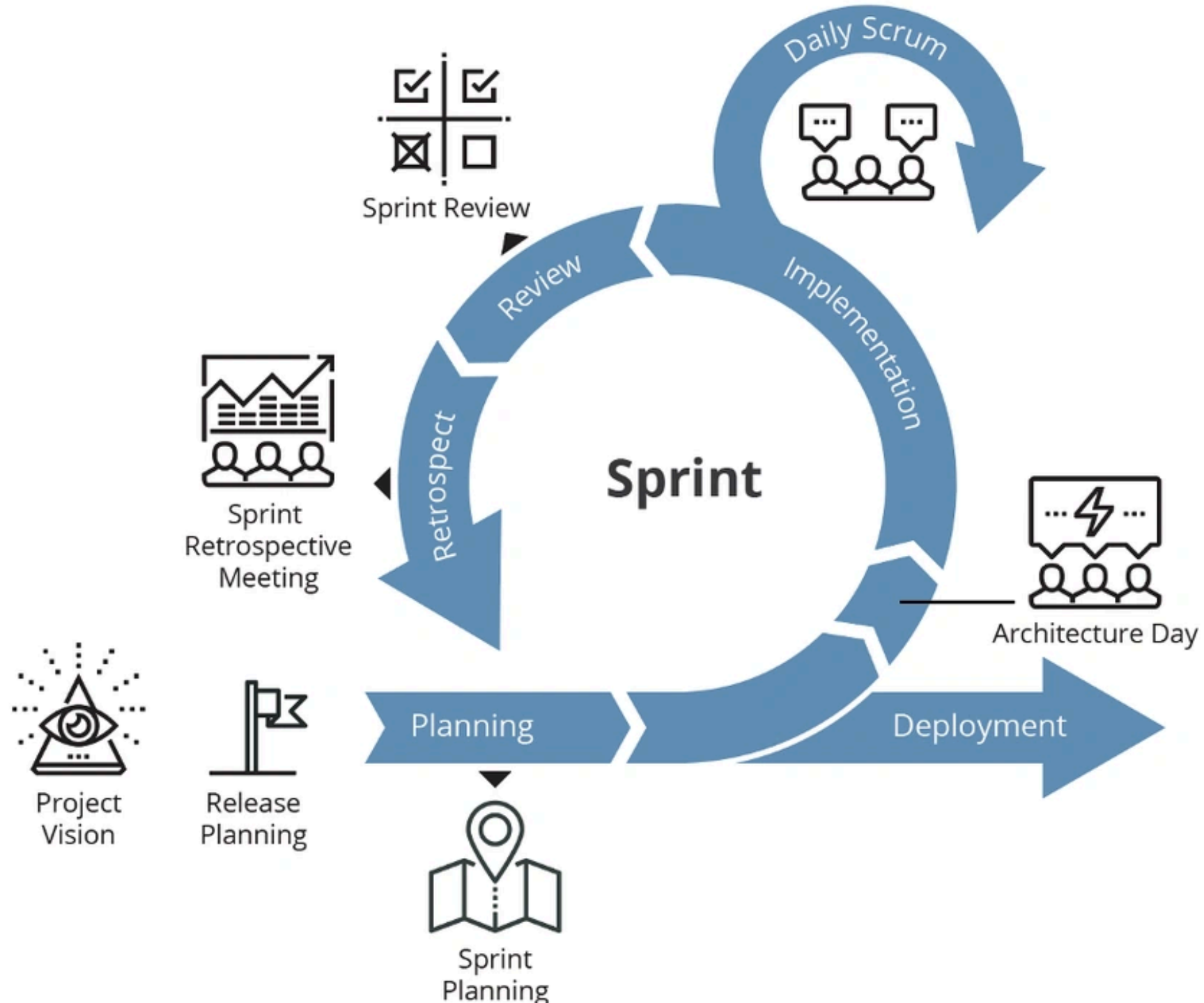
Cycle agile sans OPS



Archi logicielle: basculement des contraintes projet



“ loi de hofstadter: Il faut toujours plus de temps que prévu, même en tenant compte de la loi de Hofstadter.



Archi logicielle: qui est l'architecte ?

1. il doit déterminer une découpe en composants logiciels i.e.
 - des classes => **du code**
 2. il doit présenter des **vues** abstraites de ces composants
 3. il doit **justifier** ses vues auprès des parties prenantes
- “ profile **dev** avec des compétences en conception et communication
=> *Lead Tech avec 2 expériences pro*
=> *chef de projet + analyste de code* ”

Archi logicielle: antipatterns d'architecte

- Syndrome de la **Tour d'Ivoire**: *isolement, surplomb*
 - Titre **honorifique**: *pas d'architecture, pas de MAJ*
 - Pas de compétences dev
 - Spécialiste de la documentation :
 - **Big Design Up Front** : architecture parfaite avant tout développement
- “ dixit le *Manifeste Agile*: sys opérationnel >>> documentation
Nous privilégions le premier **MAIS** nous reconnaissons la valeur du second.

”

II. Principes d'Architecture

II.1 Pas de Liberté sans Contraintes

- les règles d'architecture vont *diminuer les degré de liberté* du code pour assurer les objectifs de l'architecture
 - simplification / évolutivité / maintenabilité

exemples historiques: les **paradigmes de code**

- *impératif* (1968 Dijkstra)
- *fonctionnelle* (1936 Church / Turing)
- *orienté objet* (1966 Dahl / Nygaard)

Programmation impérative

- sa *robustesse* et sa *perennité* tient à
 - **l'interdiction des instructions goto**
 - sauf en cas de selection => *if / then / else*
 - sauf en cas d'itération => *while / do / repeat*
 - cette contrainte permet la **décomposition fonctionnelle** *réfutable*
 - de tout module comme
- “ *module: unité logique et autonome de code qui encapsule des fonctionnalités liées.* ”

Programmation Fonctionnelle

- hérité du « calcul Lambda » cf *fonct° lambdas*
- le programme se décrit comme une **composée de transformations**
 - produisant un objet de *Sortie* à partir d'un objet d'*Entrée*

$$S = f \circ g \circ h \circ \dots (E)$$

- en particulier, dans ce paradigme,
 - *aucune variable globale ou locale ne peut être modifiée*
 - c'est le rôle des fonctions
- “ on appelle cette contrainte l'**immutabilité**

”

Programmation Orientée Objet

- historiquement, un objet a été conçu en examinant
 - le comportement de l'état d'une fonction de la *pile d'appel*
 - transféré dans le *tas* « heap »
 - => rendant les variables locales *persistantes* au delà de l'instruction *return*
- pour manipuler correctement ces variables nommées **attributs**
 - => on a dû de **contraindre les pointeurs sur les fonctions** à n'autoriser que certains déplacements

POO: encapsulation

- le contexte extérieur a accès aux **méthodes publiques**
 - pas d'accès direct aux attributs qui sont **privés** (*frontière*)
 - une méthode publique est une *interface* entre le contexte,
 - les règles / données métiers (entité) => *contrôle*
 - et l'actualisation de l'*état de l'instance* gérée par les **méthodes privées** => *sécurité*
- “ *Injection de dépendance: un objet extérieur ne doit connaître que l'interface/signature publique d'un autre objet => couplage faible* ”

```
def withdrawal(self, amount: float) -> dict:
    if 0 < amount <= self.__balance + self.__overdraft:
        self.__set_balance(-amount)
    else:
        print(f"montant invalide: {amount}")
```

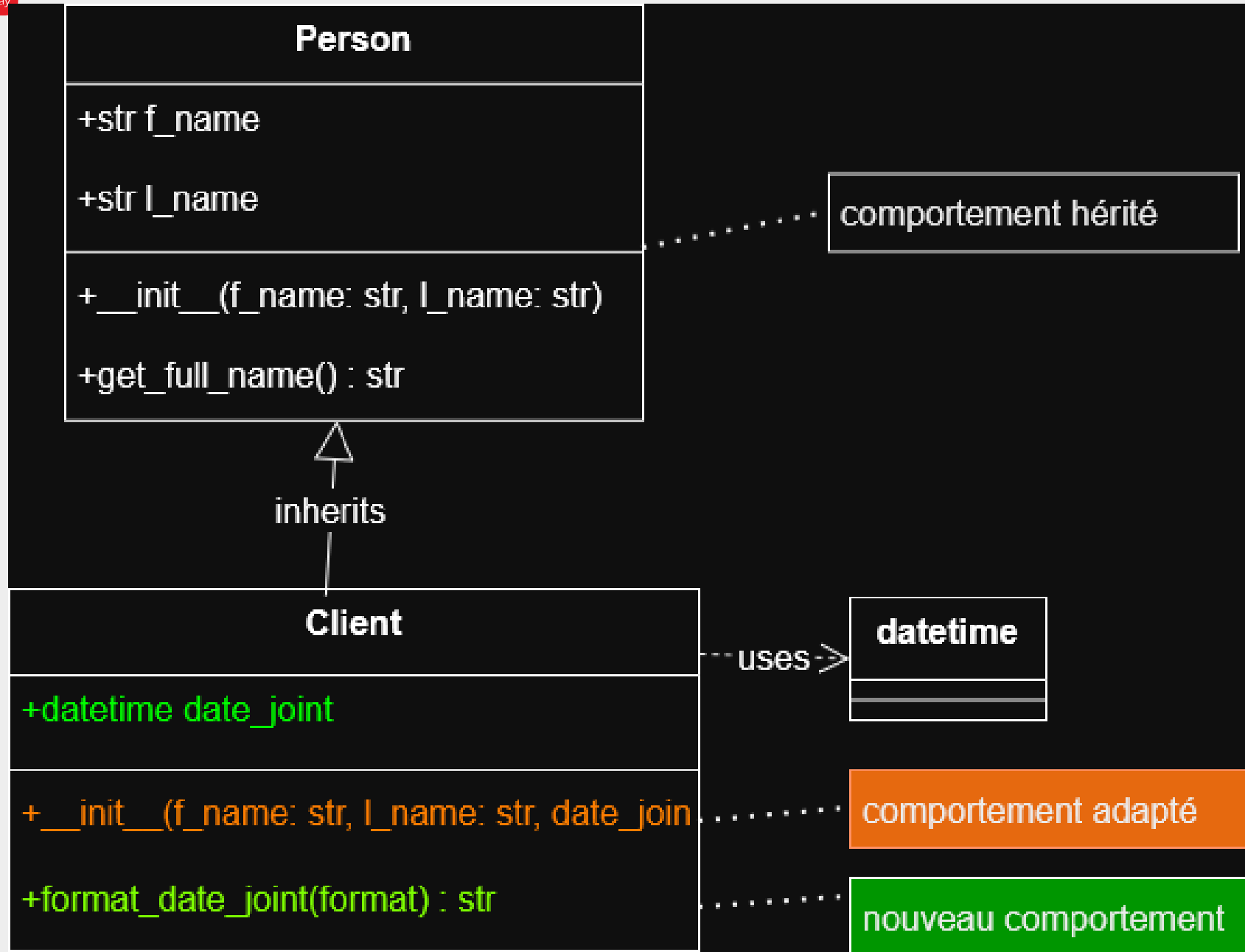
Client
+ account: Account

withdrawal

Account
-int __id
-float __balance
-float __overdraft
+__init__(_id: int, balance: float, overdraft: float)
+get_balance() : float
+withdrawal(amount: float) : dict
-__set_balance(amount: float)

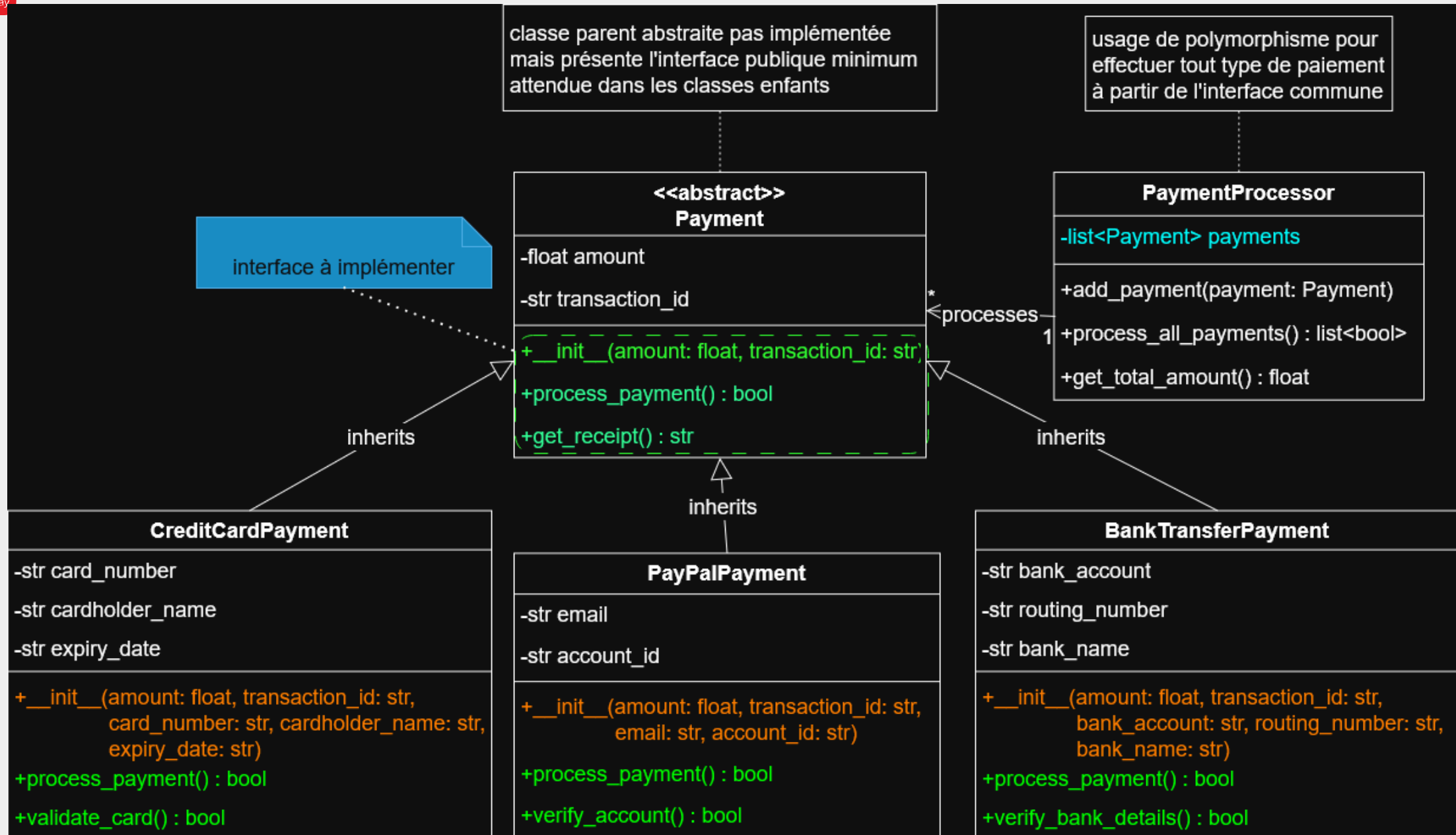
POO: Héritage

- mécanisme de **personnalisation** d'une classe / type de données
- la classe enfant est une *nouvelle classe* contenant
 - les comportements de la classe parent
 - et de nouvelles fonctionnalités
- **la classe parente n'a pas changé** => *immutabilité*
- la classe parente est partagée par l'arbre des classes enfants
=> *factorisation*



POO: Polymorphisme / Abstraction

- pour profiter au mieux de l'*injection de dépendance*
 - un objet extérieur *doit pouvoir utiliser*
 - une **interface (au sens Java)** => définit^o de signatures publiques
 - ou une **classe Abstraite** présentant les mêmes signatures
 - au lieu d'*utiliser directement* la collection de classes concrètes
- “ l'objet extérieur est dit **enfichable**, il ne dépend que d'une définition et non de classes plus spécialisées
=> **inversion de dépendance** ”



II.2 Principes S.O.L.I.D

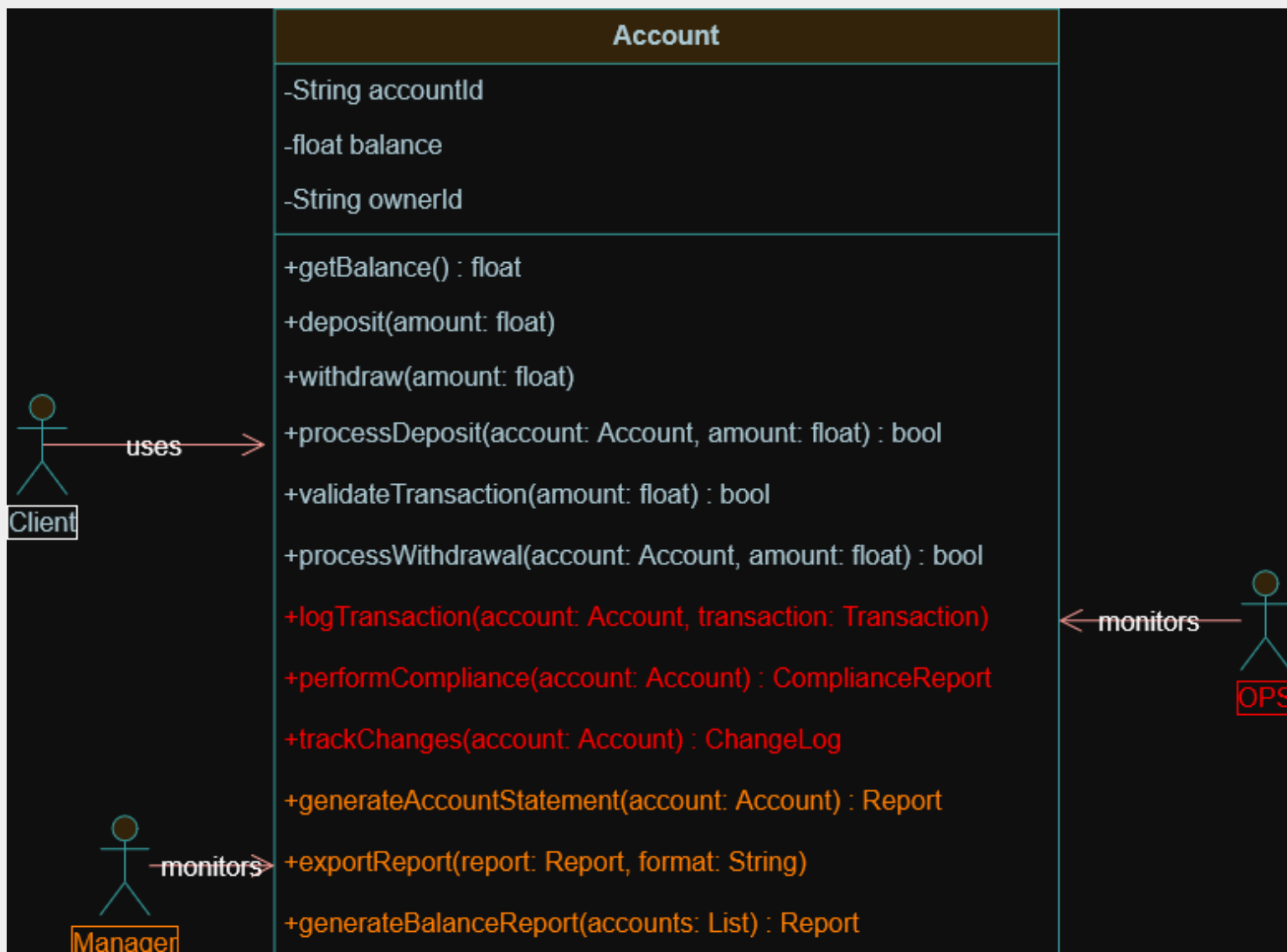
- 5 principes de Génie logiciel pour
 - organiser les fonctions et structure de données
 - dans une classe et entre classes dans un module
1. Principe de *Réponsabilité* unique « **SRP** »
 2. Principe Ouvert / Fermé « **OCP** »
 3. Principe de *Substitution* de Liskov « **LSP** »
 4. Principe de *Ségrégation* des Interfaces « **ISP** »
 5. Principe d'*Inversion* de Dépendance « **DIP** »

SRP

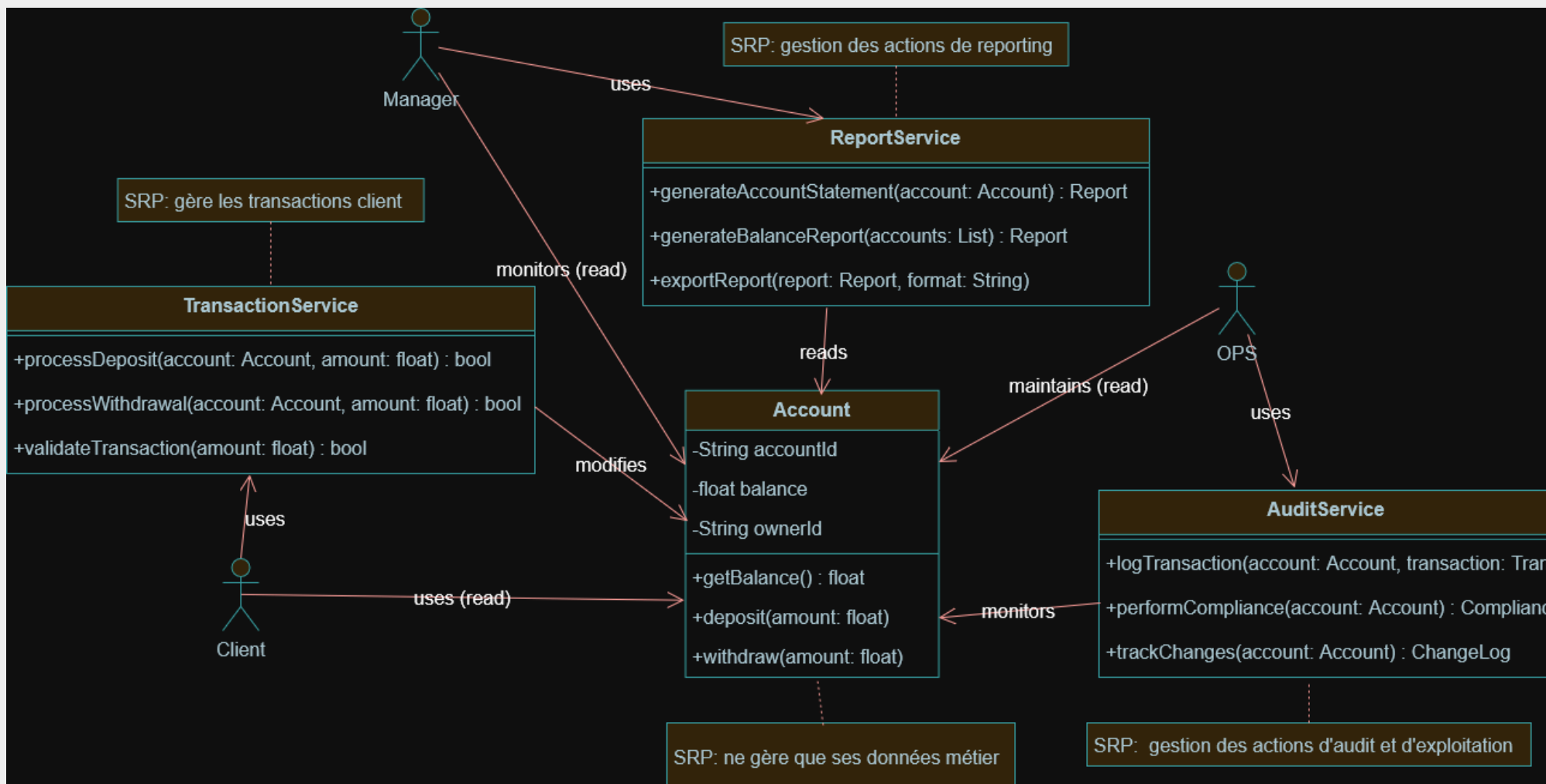
- *contre sens: la responsabilité n'est pas la fonctionnalité !*
- **la responsabilité désigne l'acteur qui interagit avec la classe**
 - cela dit, plusieurs acteurs peuvent utiliser le même *cas d'usage* unique

“ les méthodes d'une classe doivent avoir les mêmes causes de changement ”

SRP : problème

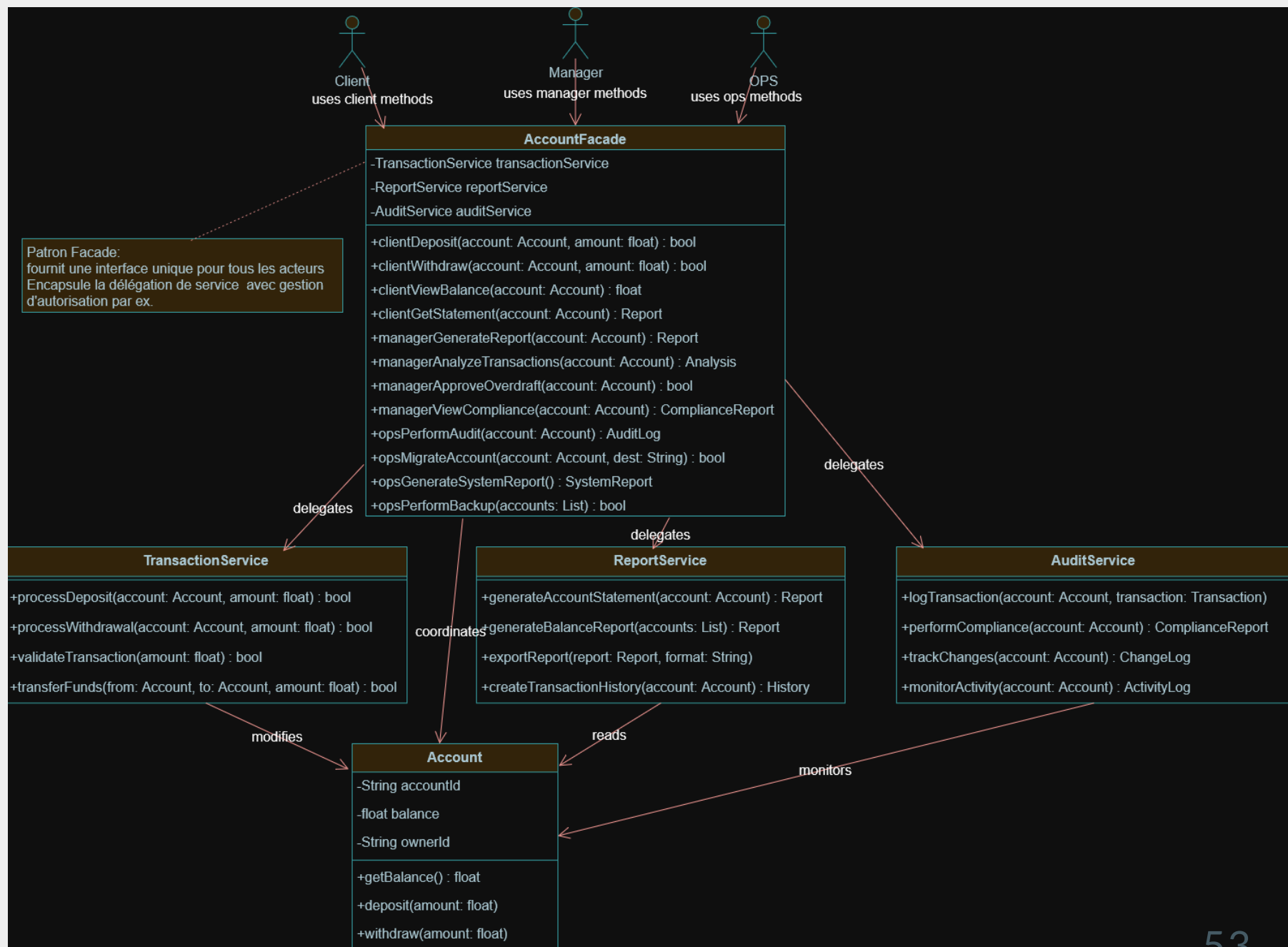


SRP : solutions



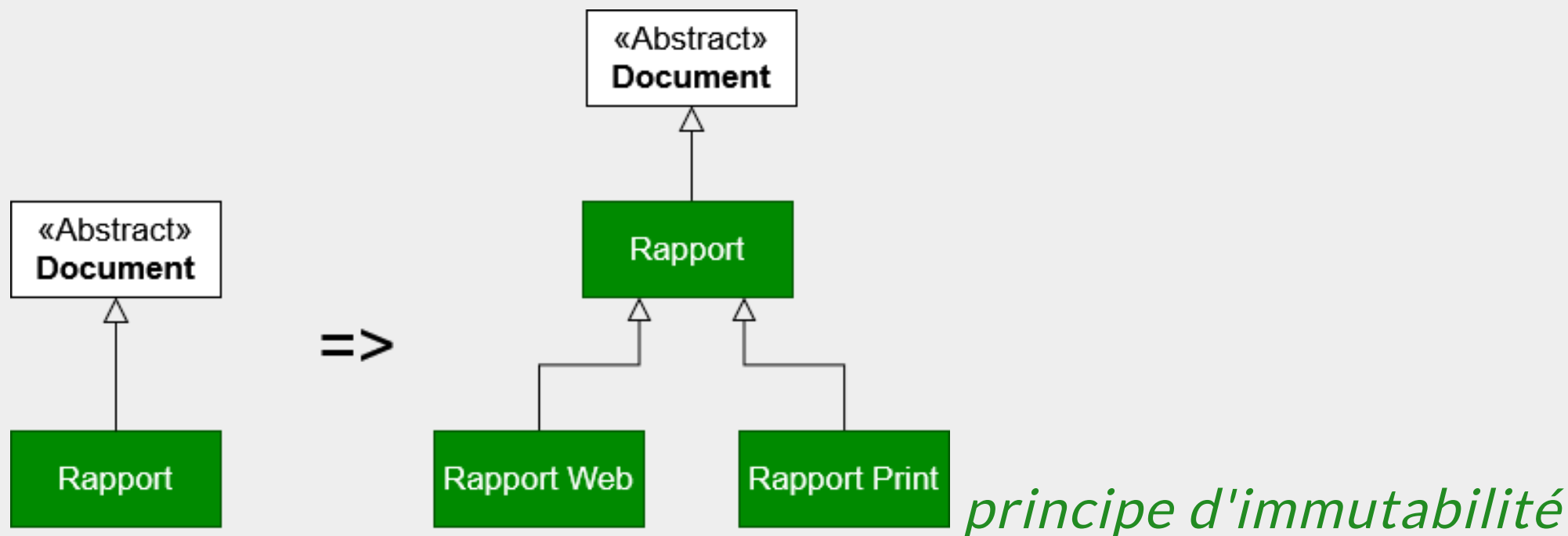
SRP : Façade

- classe utilitaire
- guichet unique



OCP: pour une classe

“ un artefact logiciel doit être ouvert en extension, mais fermé en modification ”



OCP: recommandations

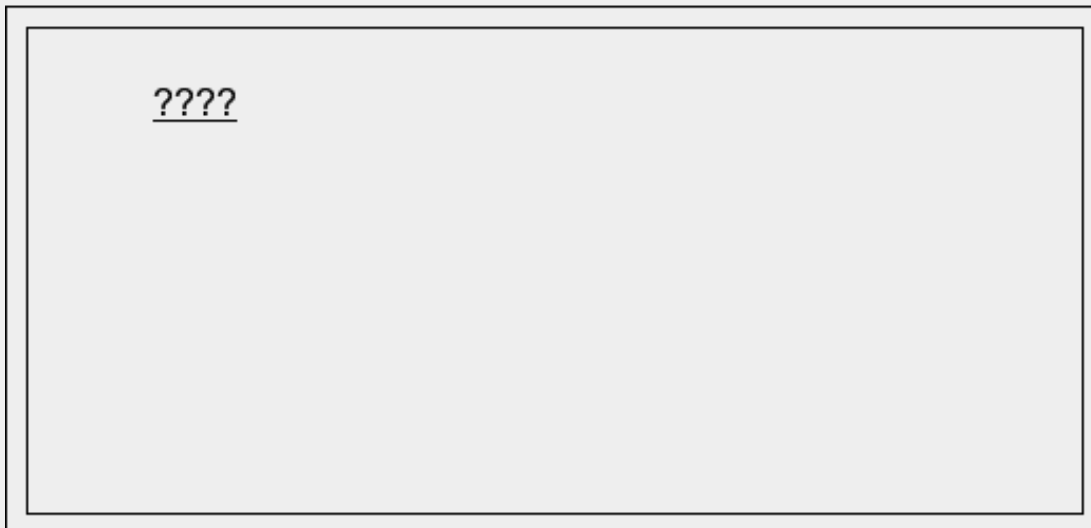
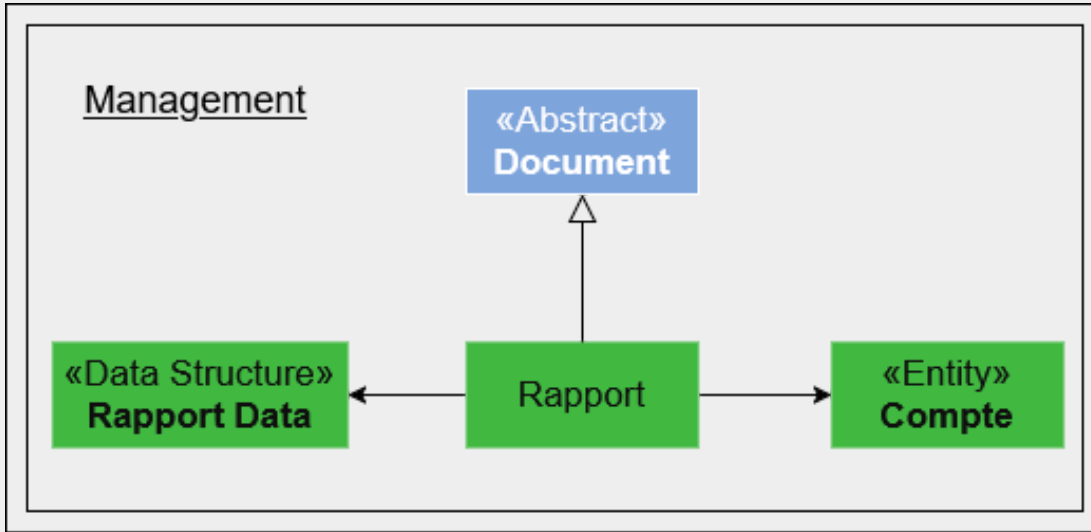
- *essayer* de ne pas hériter de classes concrètes
- ne pas surcharger « *override* » une méthode déjà concrètes
 - rendre la méthode abstraite et effectuer 2 implémentations

OCP: pour un module

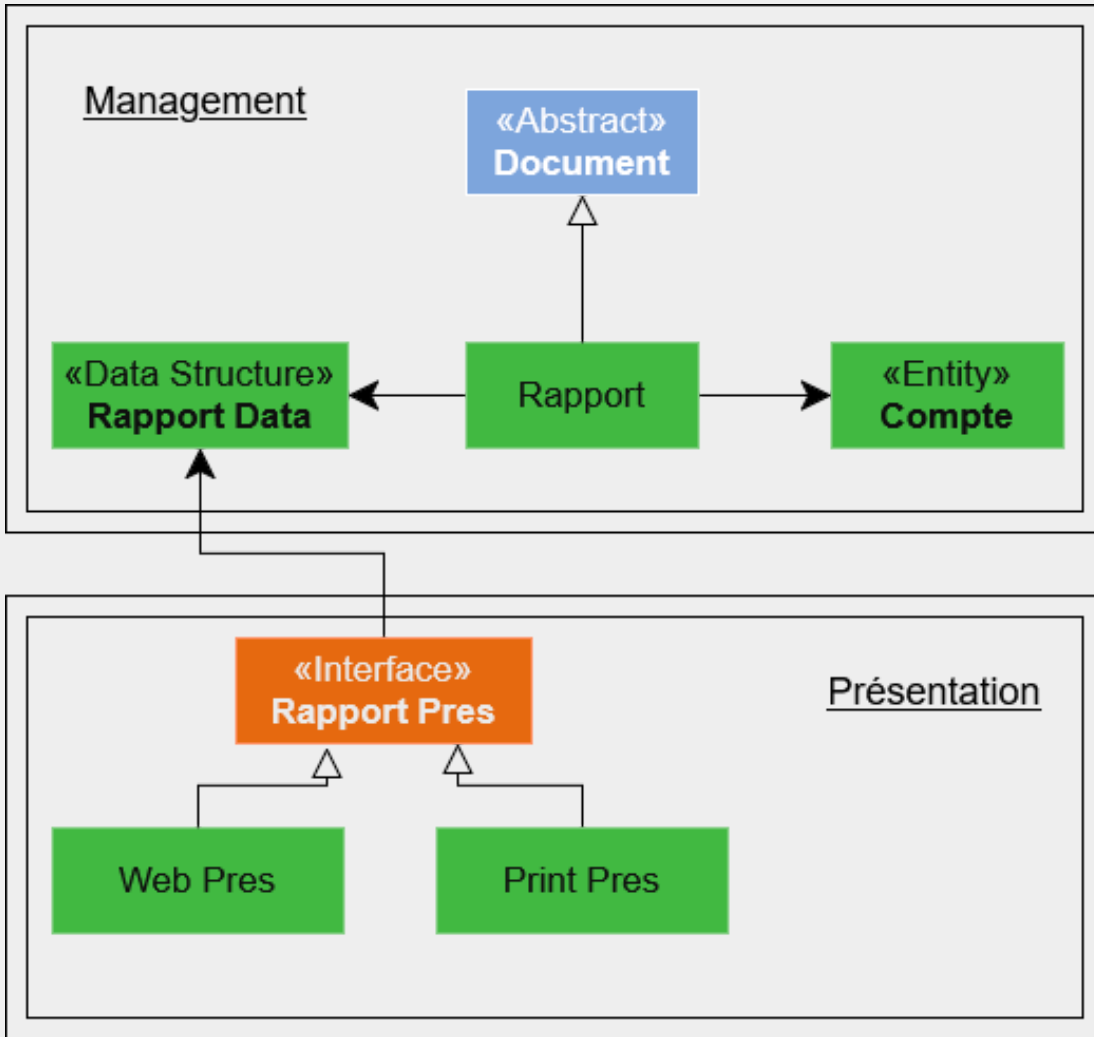
“ **un module doit être protégé des modifications des modules qui l'utilise** ”

1. les modules sont découpés selon le *SRP*
2. un module utilisé comme **dépendance**
 - ne connaît pas les détails de ceux qui l'utilise
 - **MAIS** doit présenter des *abstractions d'implémentation* i.e respecter des **interfaces**

OCP pour un module: pb



OCP pour un module: solutions ???



LSP

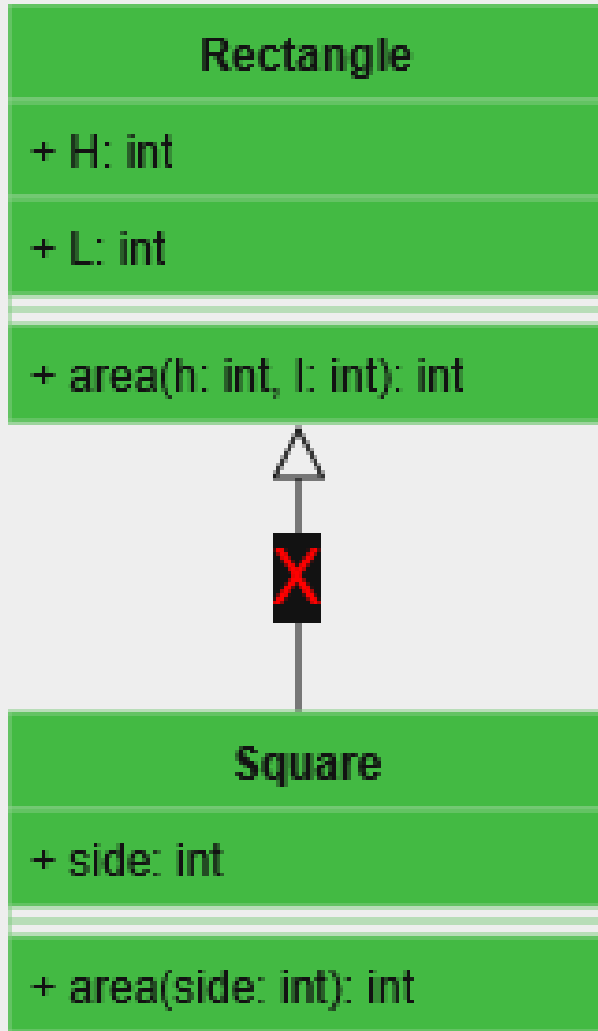
- “ Pour chaque objet *o2* du type **S**,
S'il existe un objet *o1* du type **T** tel que
Tous les programmes **P** qui sont définis en terme de **T**
dont le comportement n'est pas modifié
Lorsque *o1* est remplacé par *o2*
Alors **S** est un sous-type de **T**

Barbara Liskov (1988).

”

- **c'est le comportement des objets qui doit conditionner la substitution, pas simplement la relation d' « être »**

LSP: contre-exemple



LSP: indice de spécialisation d'une classe

- adéquation d'une classe à sa hiérarchie

$$S = \frac{M_{surchargées} \times DIT}{M_{totales}}$$

M_{surchargées} = nb méthodes surchargées

DIT = degrés dans l'arbre d'héritage

M_{totales} = nb total de méthodes

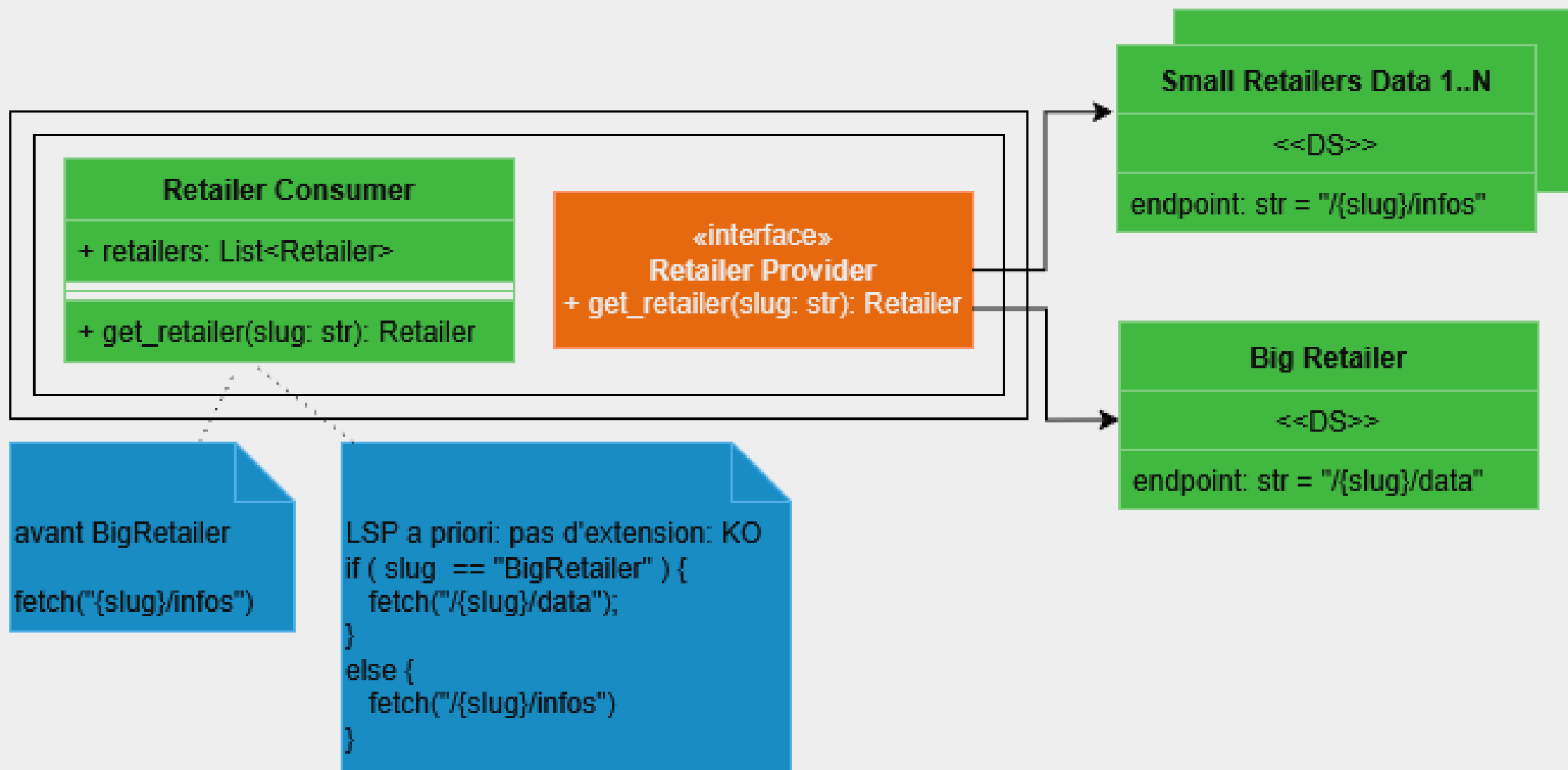
$S > 1,5 \Rightarrow$ risque de violation de LSP $\Rightarrow$ classe à restructurer

LSP: violation du principe

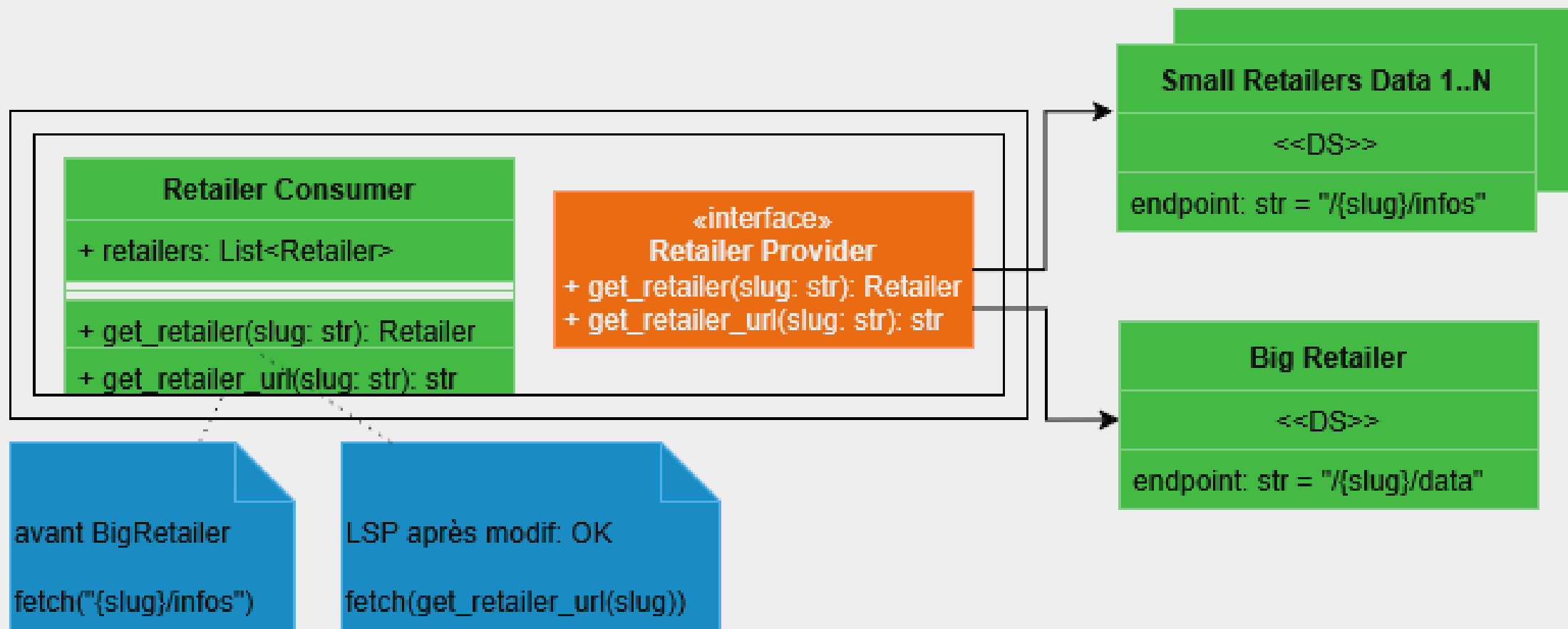
- le LSP est un *principe, non un théorème* malgré sa formulation

“ certains cas d'usage, notamment liés à des *considérations business*, permettre de **changer un comportement pour substituer un objet initialement non aligné** ”

LSP: pb

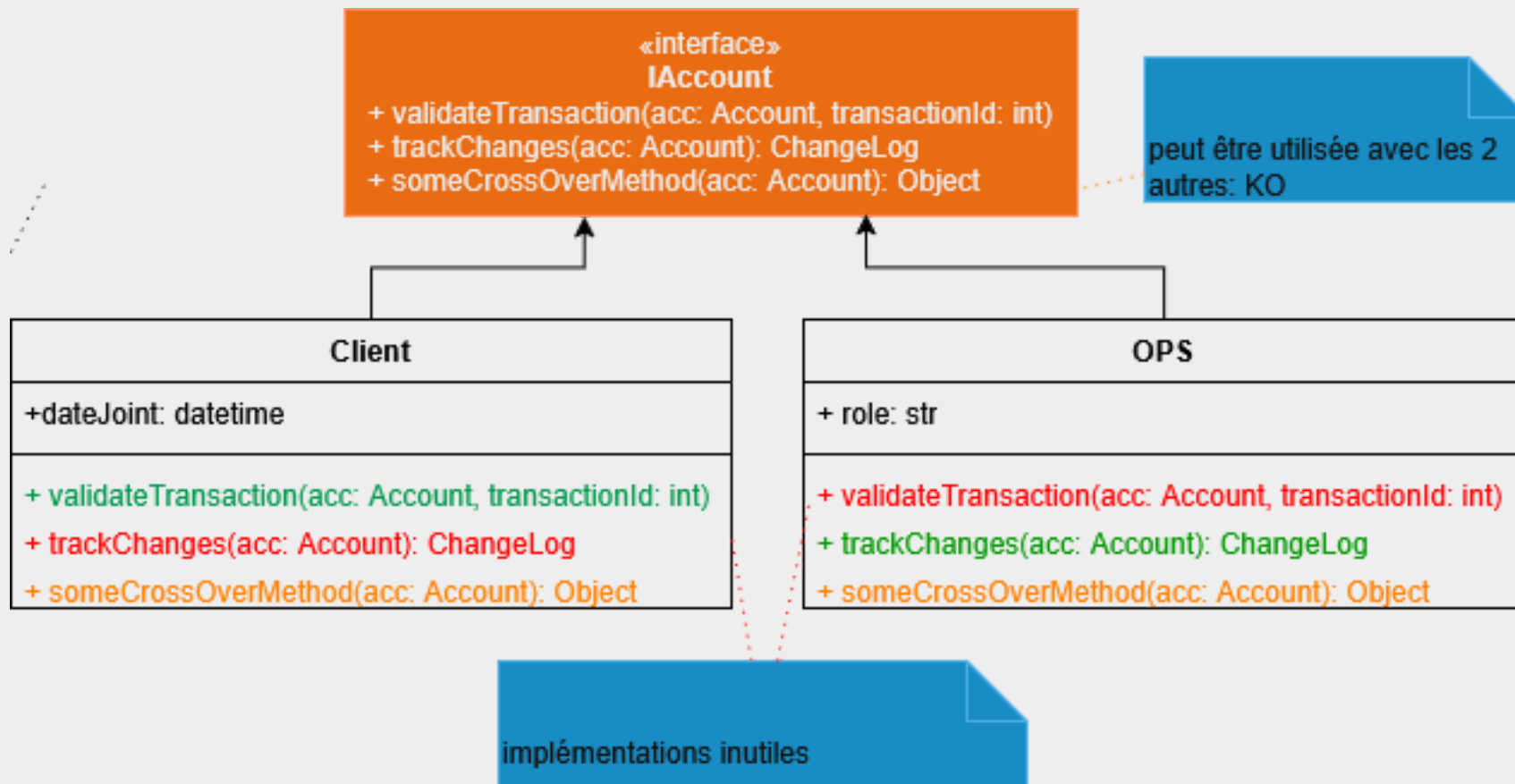


LSP: solutions ???



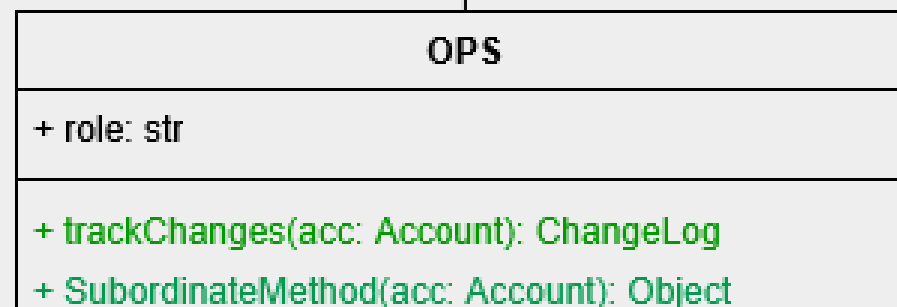
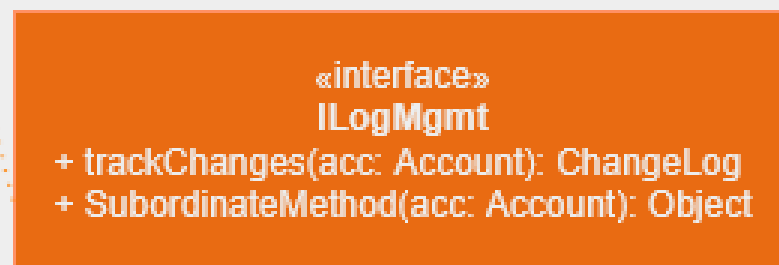
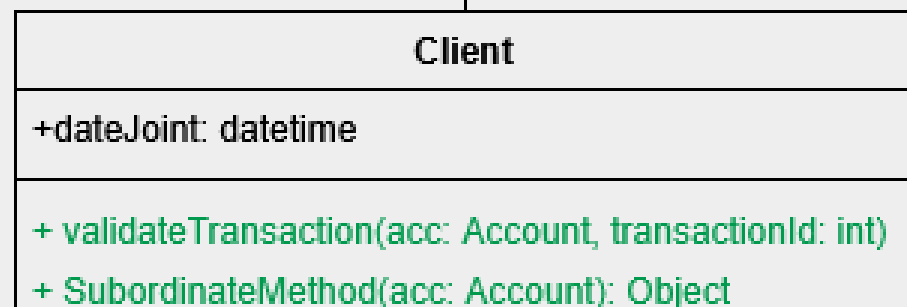
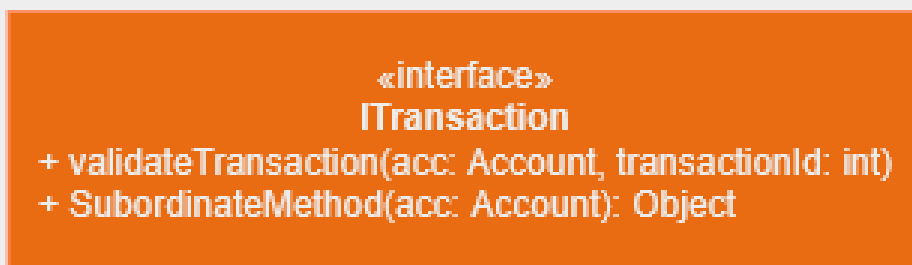
ISP

“ il est toujours dangereux de rendre un code dépendant de modules qui contiennent plus d'éléments que nécessaire ”



ISP: solutions ???

duplication acceptable dans des flux métiers différents
et probablement temporaire



DIP

- une classe *cliente* doit **définir et utiliser des abstractions**
- une *dépendance* doit **implémenter des abstractions**
- les primitives (*String*, *Integer...*) n'ont pas besoin d'abstraction
- les interfaces bâties sur un nouveau sujet sont souvent volatiles
 - **mais les interfaces devraient embarquer un savoir faire reconnu !**

“ *les systèmes les plus adaptables sont ceux qui présentent des abstractions* ”

DIP: violation du principe

- il y a toujours des dépendances concrètes
- *exemple:* pour la création des instances concrets
 - utiliser le patron « *Fabrique Abstraite* »

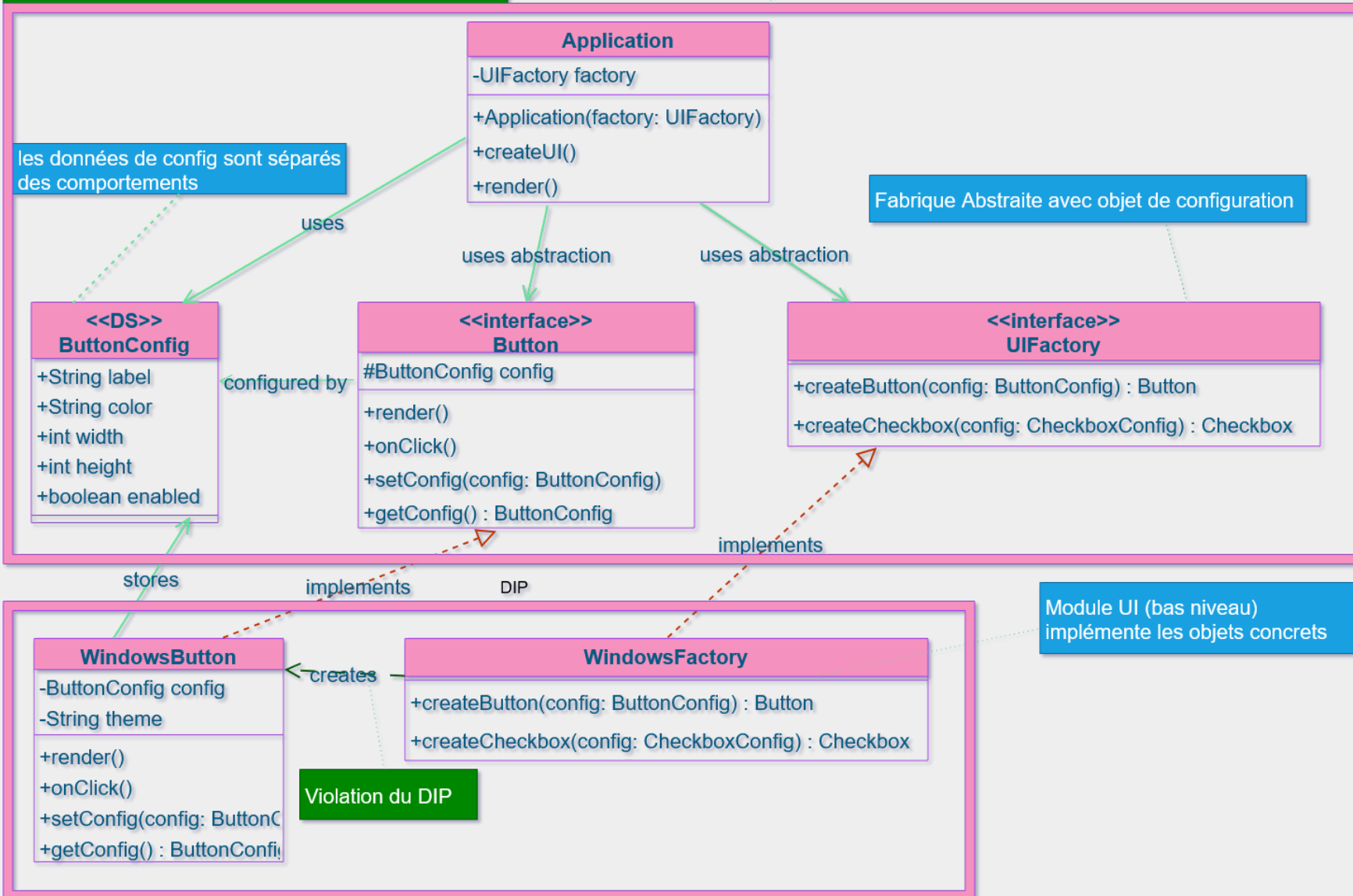
Principe d'inversion de dépendances

1. les modules haut niveau dépendent d' abstractions
2. les objets de config sont passés dans la Fabrique
3. les détails dépendent d'abstractions

Module Principal main (haut niveau)
ne dépend que d'abstractions (UIFactory, Button)

les données de config sont séparés
des comportements

Fabrique Abstraite avec objet de configuration



II.3 Cohésion des composants

- un composant est une **unité de déploiement**
 - le *+ petit ensemble déployable en autonomie* dans un système
 - un assemblage de classe ou modules *en cohésion*
 - quels sont les principes favorisant cette cohésion ?
 - Principe d'Equivalence Réutilisation et déploiement « **REP** »
 - Principe de Clôture Commune « **CCP** »
 - Principe de Réutilisation Commune « **CRP** »
- “ ces principes s'utilisent sur les *composants ET les modules* ”

REP

- une unité de *déploiement réutilisable*
 - est une *archive* souvent exécutable (.jar, .war, .gem, .exe...)
 - gratifiée d'une *version* « *major.minor* » (tag Git)
 - et assortie d'*informations d'usage*
=> aussi appelée « **release** »
- ce principe demande de **déployer des composants comme releases**
- “ les outils réalisant ce principe sont les *gestionnaires de paquets*: Maven, Composer, PIP, NPM, ...
moteurs d'Intégration & Déploiement Continus: Gitlab, Jenkins ... ” 71

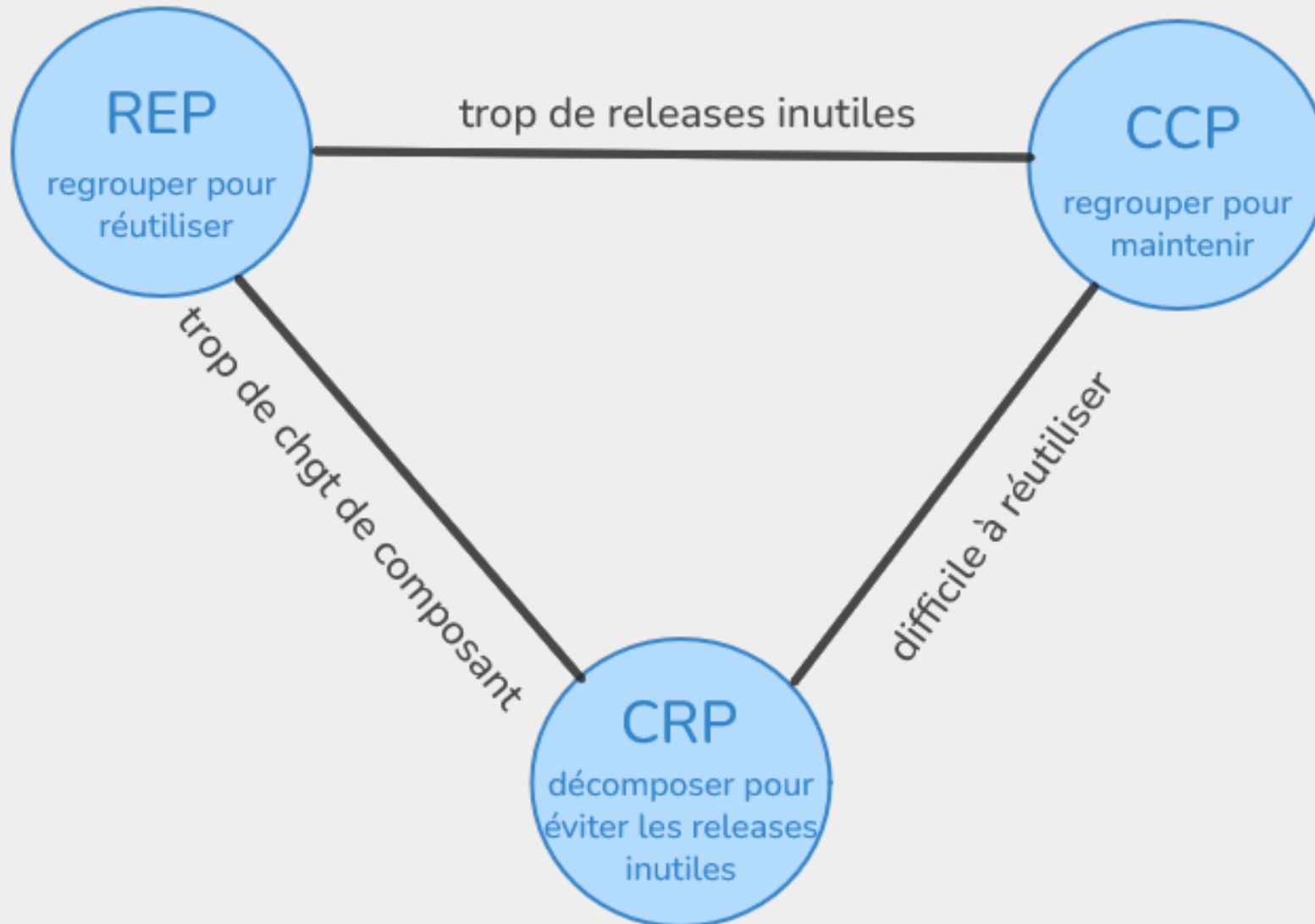
CCP

- “ Il faut regrouper dans un **même composant** les classes qui évoluent pour les **mêmes raisons et au même rythme**.
contraposée: Il faut distribuer dans des composants différents les classes qui évoluent à des moments différents et pour des raisons différentes ”
- *extension* du **SRP** aux composants
 - *relation* avec le **OCP**: les éléments d'un composant devraient avoir les mêmes types d'extension
- “ *Astuce: manipuler / observer le Modèle Conceptuel de Données (MCD)* ”

CRP

- “ **Il ne faut pas obliger les utilisateurs d'un composant à dépendre d'éléments dont ils n'ont pas besoin.** ”
- si un composant utilise (*dépend* d') un autre
=> donc au - une classe directement et / ou d'autres transitivement
=> *mais dépend également de toutes les autres classes !*
 - à la moindre modification de la dépendance
 - même si l'utilisateur n'a pas été impacté par la modif
=> il faut re *[compiler + tester + déployer]* **(utilisateur + dép.)**
- “ extension de l'ISP: **ne pas regrouper des éléments qui vont faire dépendre inutilement les utilisateurs** ”

tension entre principes



II.4 Couplage des composants

- après avoir conçu des composants selon les *principes de cohésion*
- on doit aussi considérer les **relations entre composants**
- quels sont les principes simplifiant ces relations ?
 - Principe des Dépendances Acycliques « **ADP** »
 - Principe des Dépendances Stables « **SDP** »
 - Principe d'abstraction stable « **SAP** »

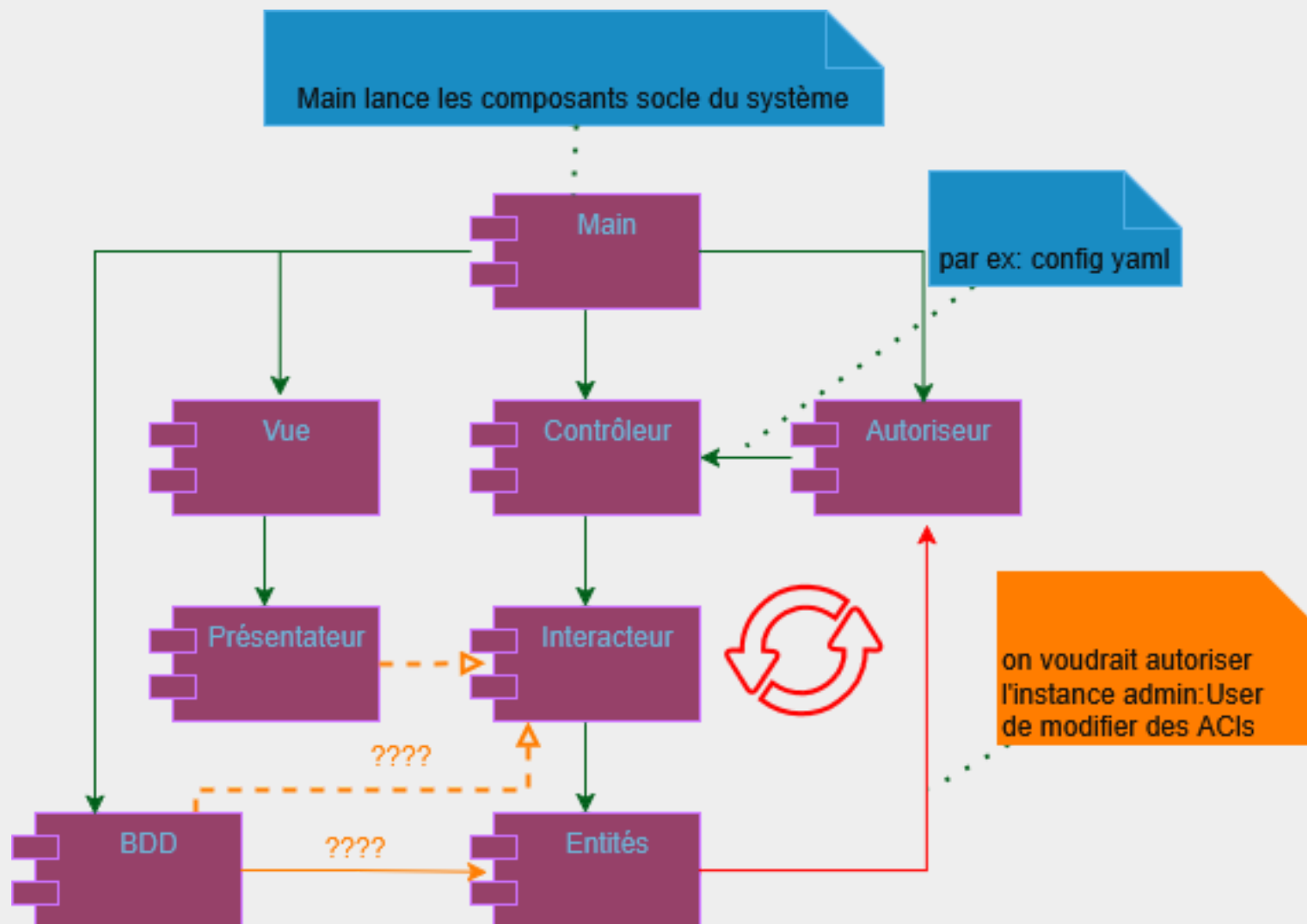
ADP

- un système se présente comme un graphe orienté contenant
 - des composants (*noeuds*) et leurs dépendances (*arcs*)
- un **cycle** dans ce graphe représente
 - un *jeu de dépendances réciproques* direct ou transitif
=> source de **problèmes de maintenance et d'évolution**

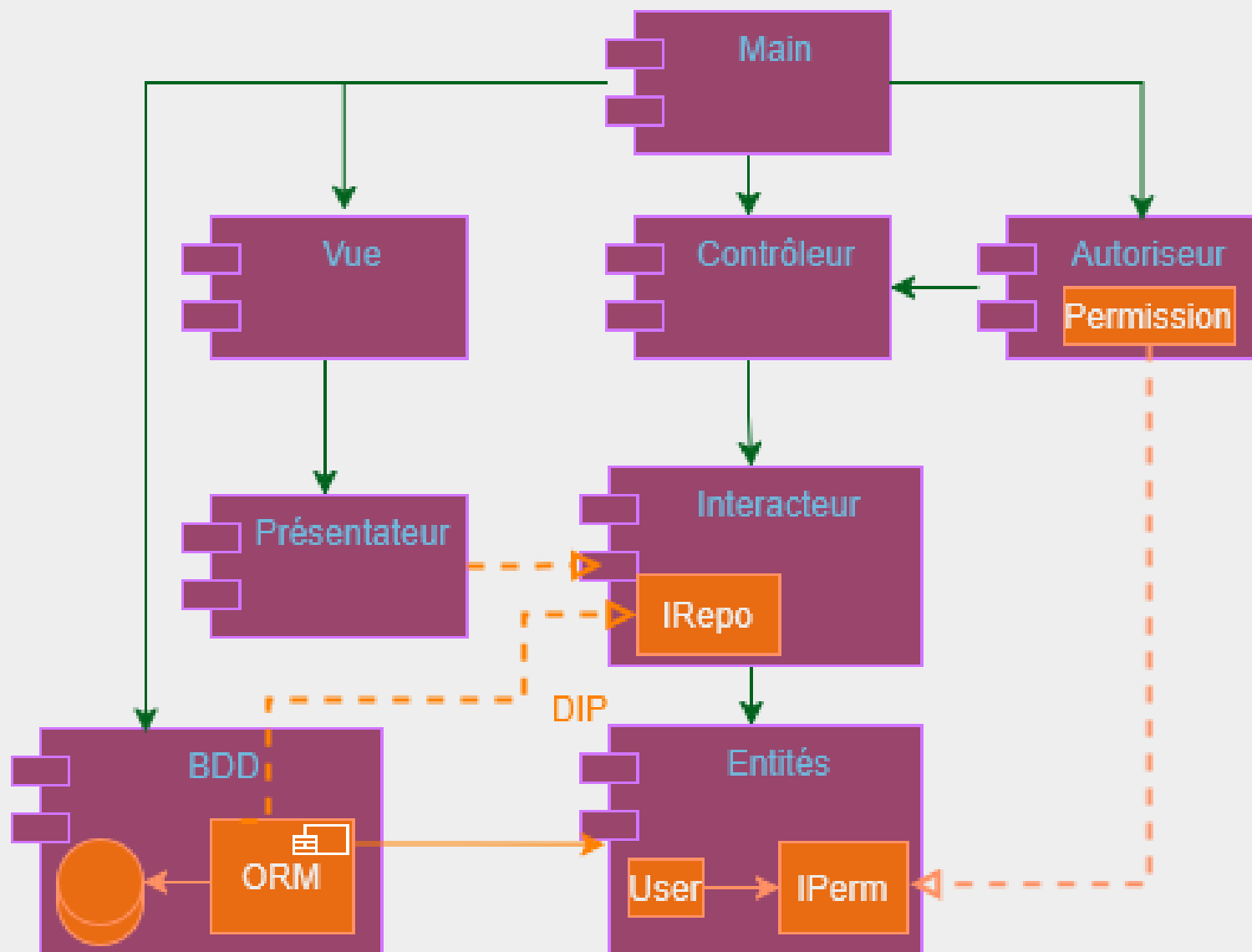
“ **un système doit supprimer les cycles dans son graphe de dépendances**

”

ADP: cas emblématique, le méta-modèle MVC



ADP: solution du MVC ???



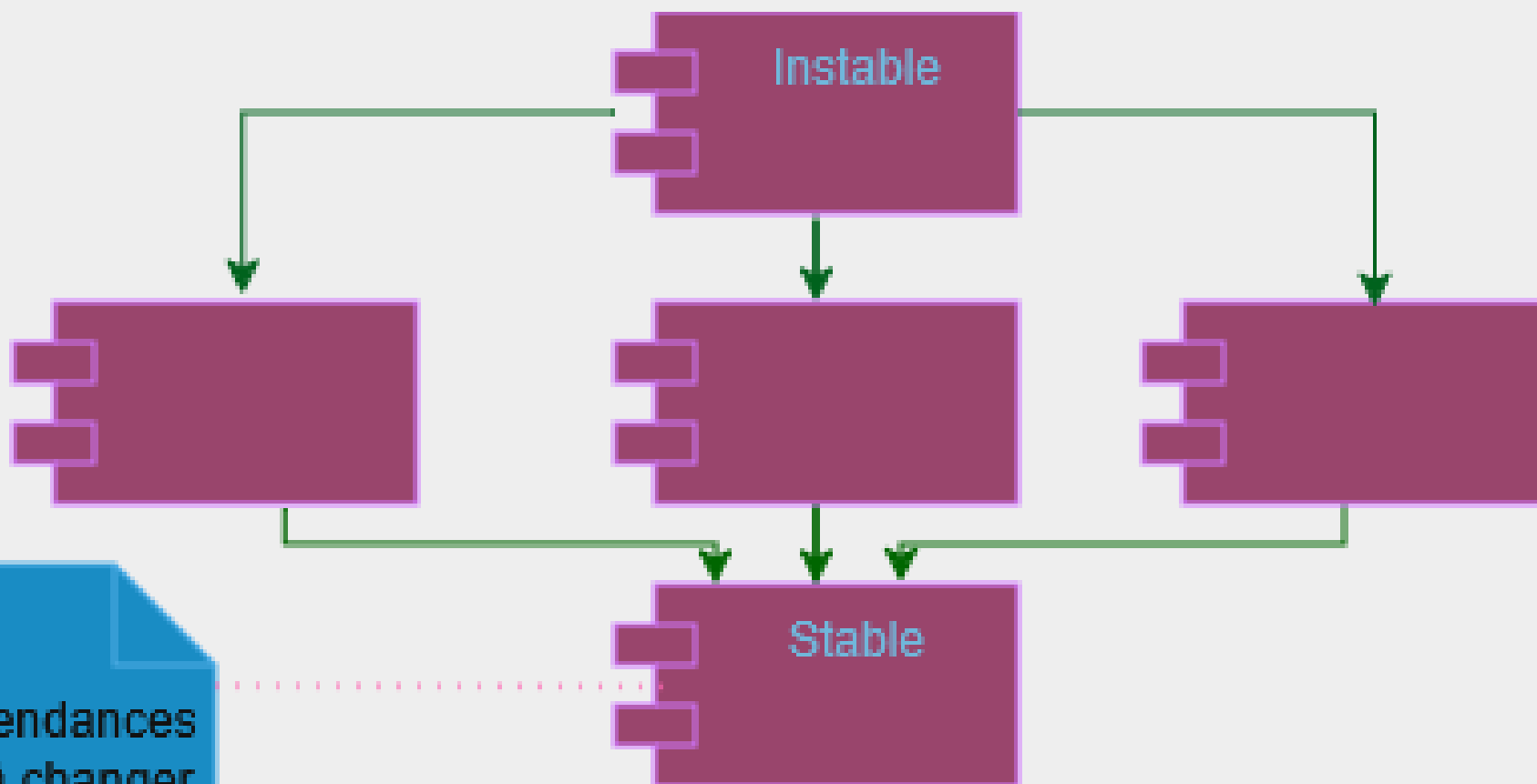
SDP

- un système n'est jamais statique
 - on considère 2 types de composants
 - **volatiles**: qui *changent souvent*
 - **stables**: qui sont *difficiles à changer*
- REM: volatile & stable = pb d'architecture

“ **Principe: les dépendances doivent être créées dans le sens de la stabilité**

”

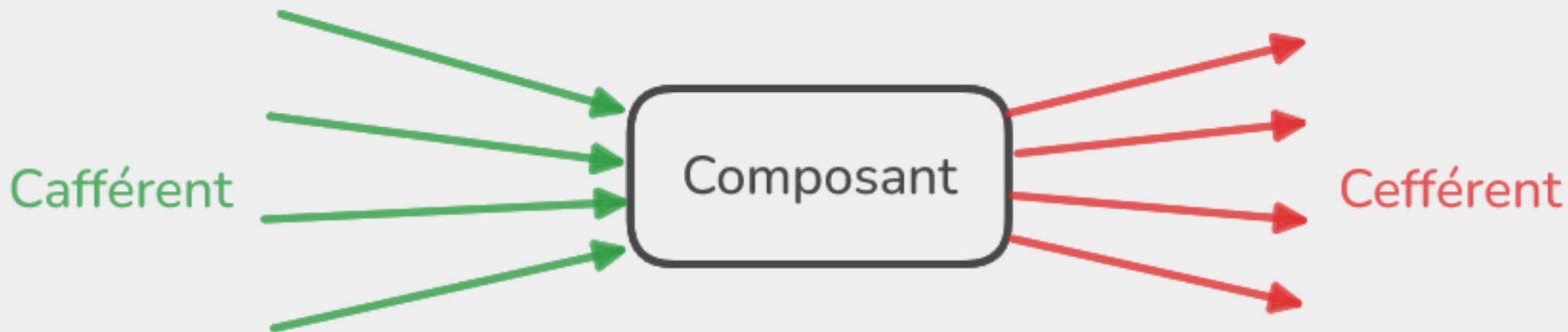
SDP: cas remarquables



bcp de dépendances
=> difficile à changer

SDP: coefficient d'Instabilité

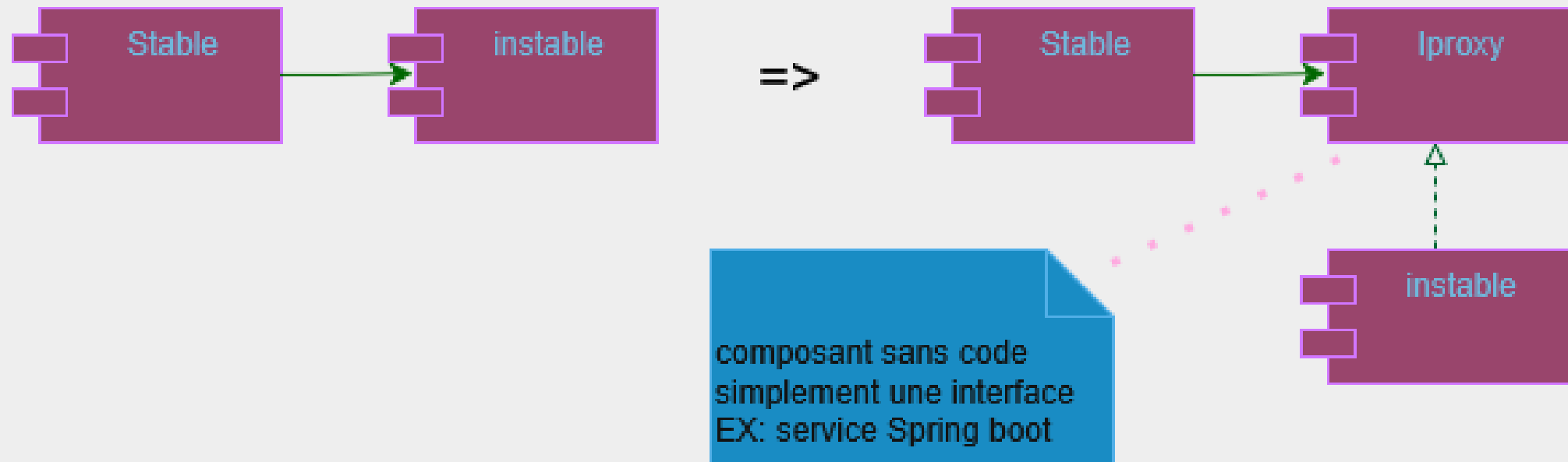
$$I = \frac{C_e}{C_a + C_e}$$



- $C_e = 0 \Rightarrow I = 0 \Rightarrow$ stable
- $C_a = 0 \Rightarrow I = 1 \Rightarrow$ instable

SDP: explication du principe

- quand un composant A est plus stable qu'un composant B
- le fait que A utilise B rend B plus stable
=> donc rend le système *plus stable*, donc *plus difficile à modifier*
- on utilise le **DIP** pour que
 - A utilise une interface ou autre abstraction
 - et B implémente l'interface



SAP

- un composant est dit « **haut niveau** » s'il contient une logique centrale et essentielle pour l'organisation
 - des *entités* => règles métier critiques (cf MVC)
 - des *cas d'usage* => flux métier critiques (cf MVC)
 - ces composants sont **naturellement stables**
 - pour favoriser une certaine souplesse de modification
 - => ces composants doivent utiliser **OCP**
 - => comprendre **le plus d'abstractions !**
- “ **Un composant doit être autant abstrait qu'il est stable** ”

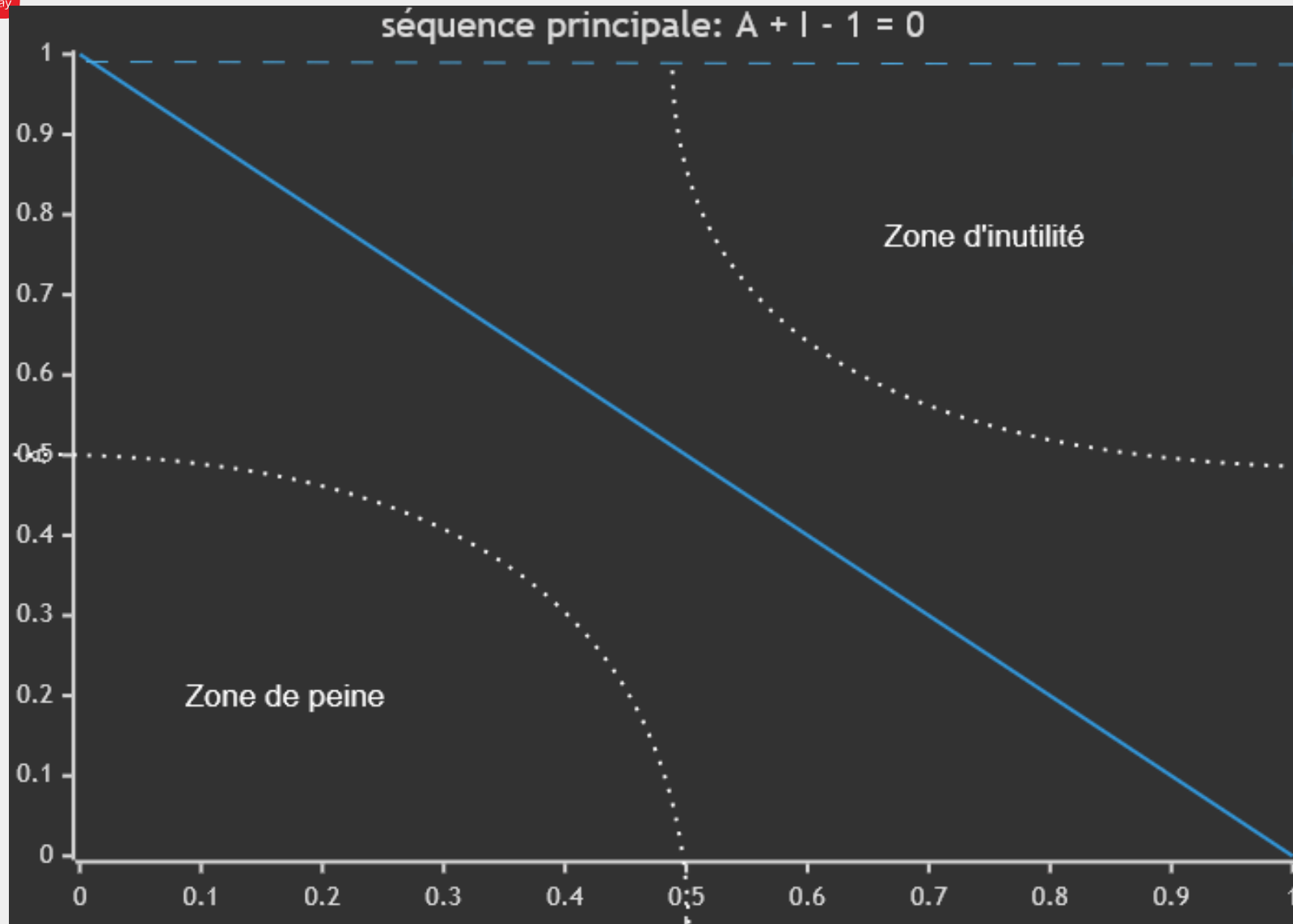
SAP: indice d'abstraction d'un composant

$$A = \frac{Na}{Nc}$$

- Na = nb de classe abstraite ou interface
- Nc = nb de classes
- $A = 1 \Rightarrow$ composant abstrait
- $A = 0 \Rightarrow$ composant concret

SAP: Séquence principale

- on peut présenter le couple **(I, A)** pour les composants du système
- avec un *graphe XY : abscisses en I [0, 1] / ordonnées en A [0, 1]*
- la droite $A = 1 - I \iff \mathbf{A + I - 1 = 0}$ est *la séquence principale*
 - représente la zone idéale d'un composant
- un composant *stable et peu abstrait* est
=> dans la **zone de peine, difficile à modifier**
- un composant *instable et abstrait* est **inutile ou inusé** car
 - *ses abstractions ne sont pas implémentées*



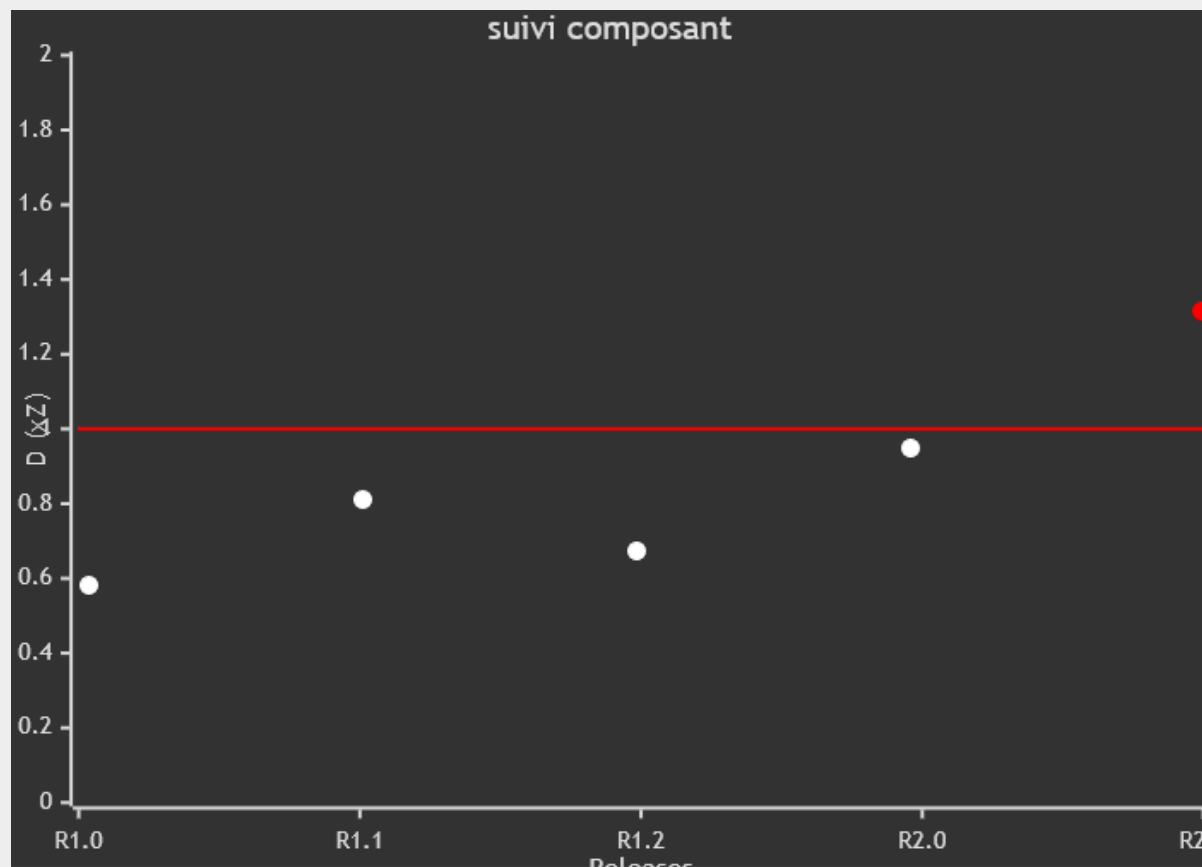
SAP: distance à la séquence principale

- pour chaque composant de la distribution statistique des composants du système on peut calculer
- la *distance à la séquence principale*:

$$D = | A + I - 1 |$$

- la *moyenne* et *l'écart type* **Z** de la distribution en A(I)
- “ les composants avec **D > Z** sont considérés comme aberrants ”

SAP: suivi des composants



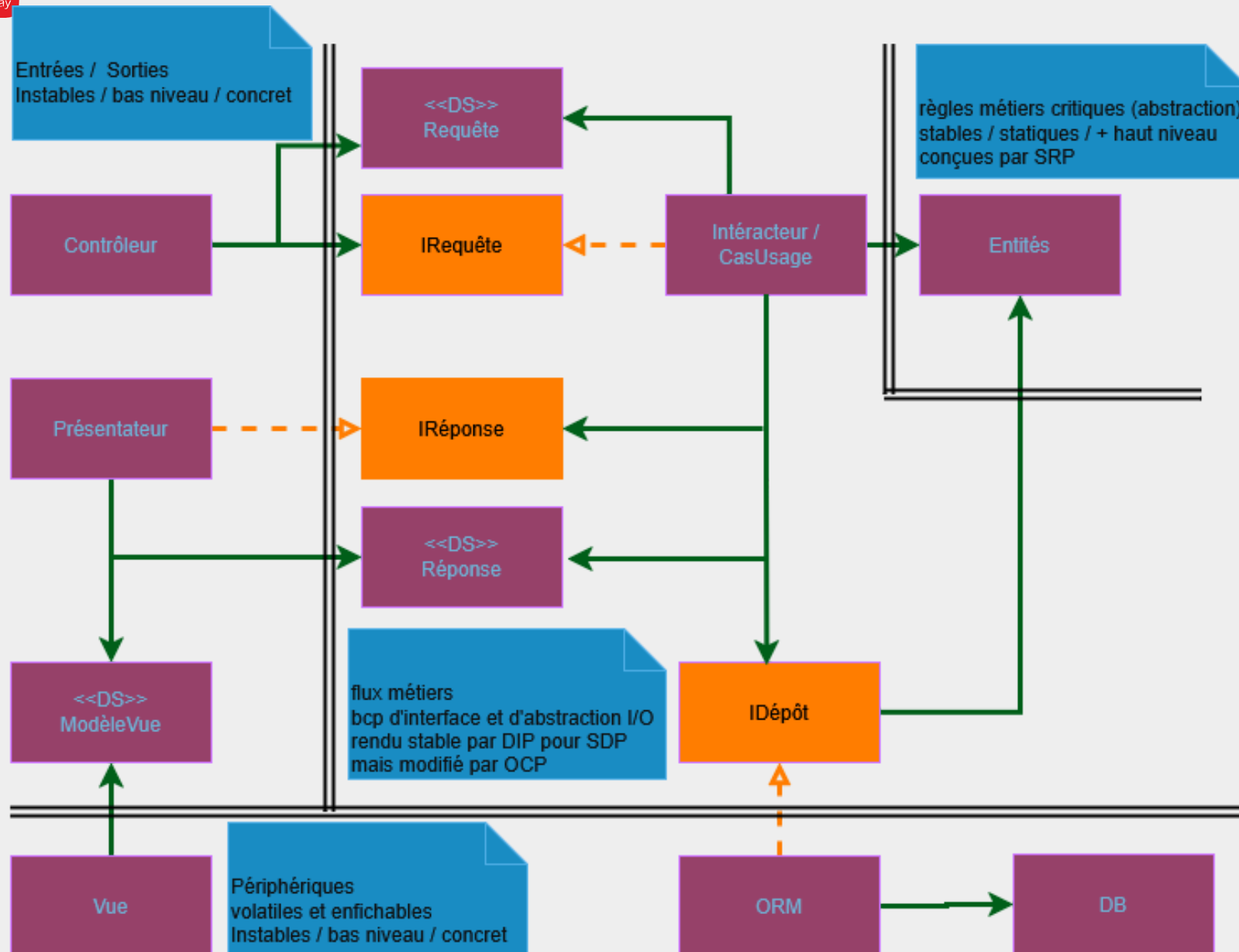
“ outil possible

”

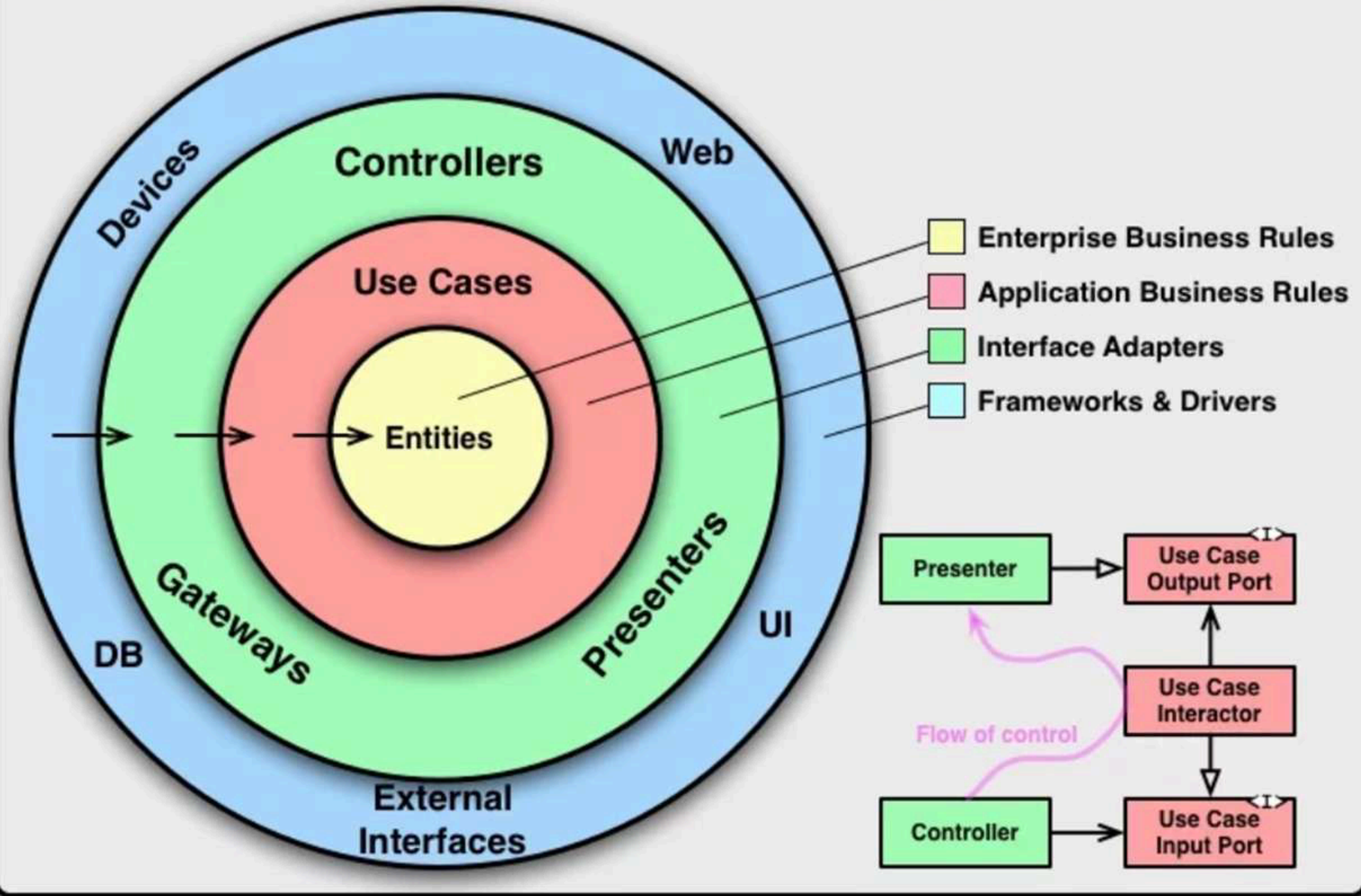
II.5 Synthèse: Architecture Propre

- la composition des principes précédents dessinent une
 - **structure de dépendances générique: Architecture propre**
- considérer les *niveaux / frontières* entre catégories d'éléments
 - bas niveau / volatiles / instable / concret
 - haut niveau / statique / stable / abstrait
- considérer le **sens général des dépendances vers les catégories de haut niveau**

“ *les niveaux ne sont pas des couches (tiers) ni des services !* ”



The Clean Architecture



II.6 Domain-Driven Design DDD

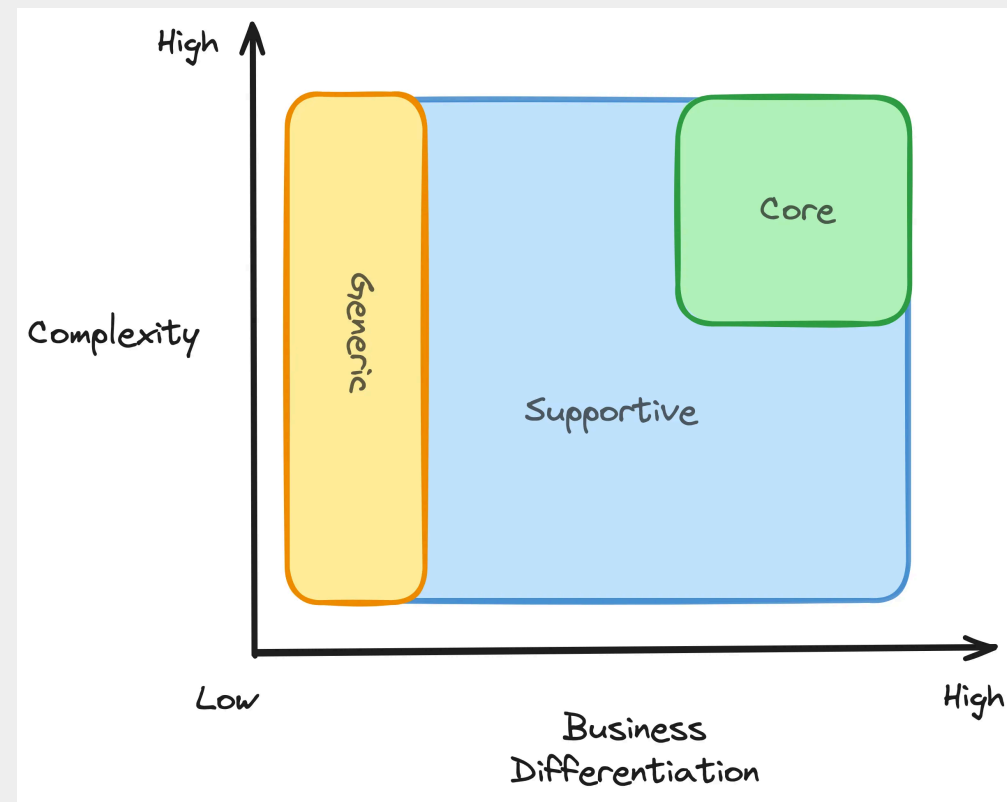
“ *pattern d'architecture logicielle*
basé sur les principes de l'architecture propre avec un
focus sur le **domaine métier** et la **découverte des entités** ”

DDD: vocabulaire ubiquitaire

- **Domaine**: ensemble des connaissances et activités d'une organisation
 - **sous-domaine**: subdivision du domaine, regroupant des concepts et activités spécifiques
 - **expert métier**: personne qui possède une connaissance approfondie d'un domaine ou sous-domaine
- “ *l'architecte doit collaborer avec les experts métier pour tablier le vocabulaire ubiquitaire à utiliser également dans le code source, par sous-domaine* ”

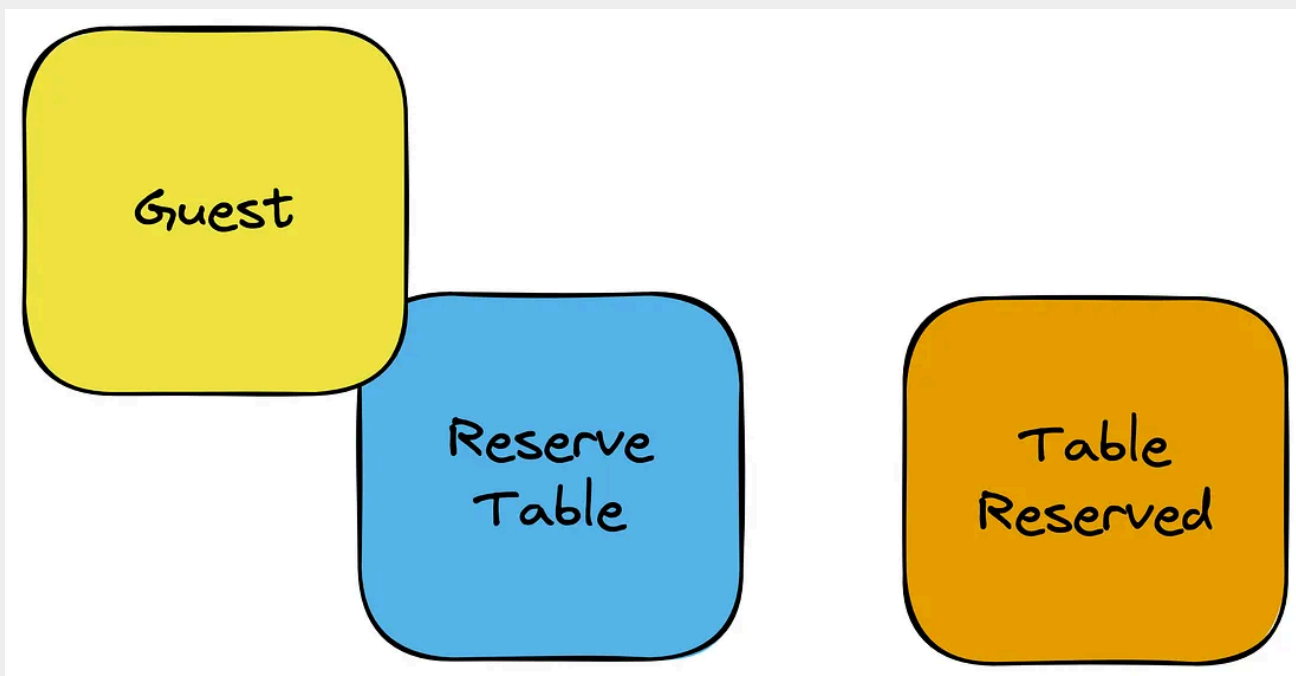
DDD: typologie de sous-domaines

- **core**: raison d'être de l'organisation, source de différenciation
- **supporting**: supporte le core, mais pas différenciant
- **generic**: pas différenciant, souvent externalisé



DDD: Event Storming

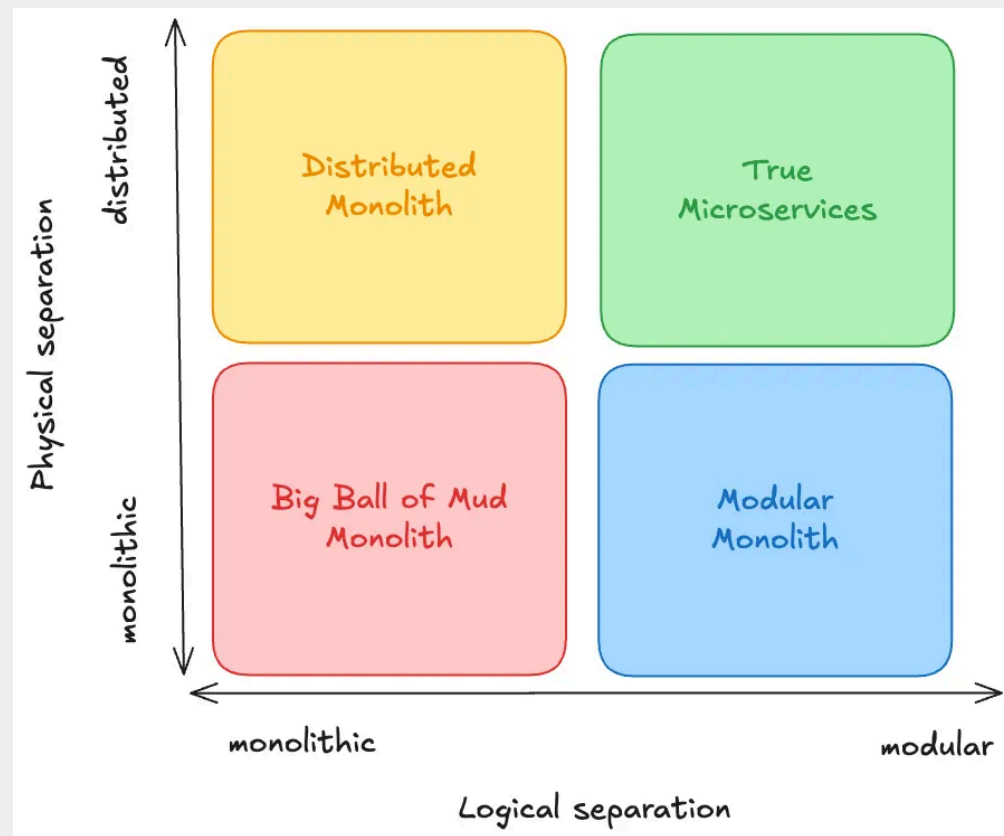
- **atelier collaboratif** pour découvrir l'agencement des *acteurs, des événements et des actions voire des entités* dans un sous-domaine



“ à entériner avec un diagramme de flux métier (BPMN) !

DDD: Bounded Context

- **découpage en sous-domaines** par les *changements de vocabulaire*
- candidats pour des *modules ou composants* logiciels autonomes



DDD: Context Maps

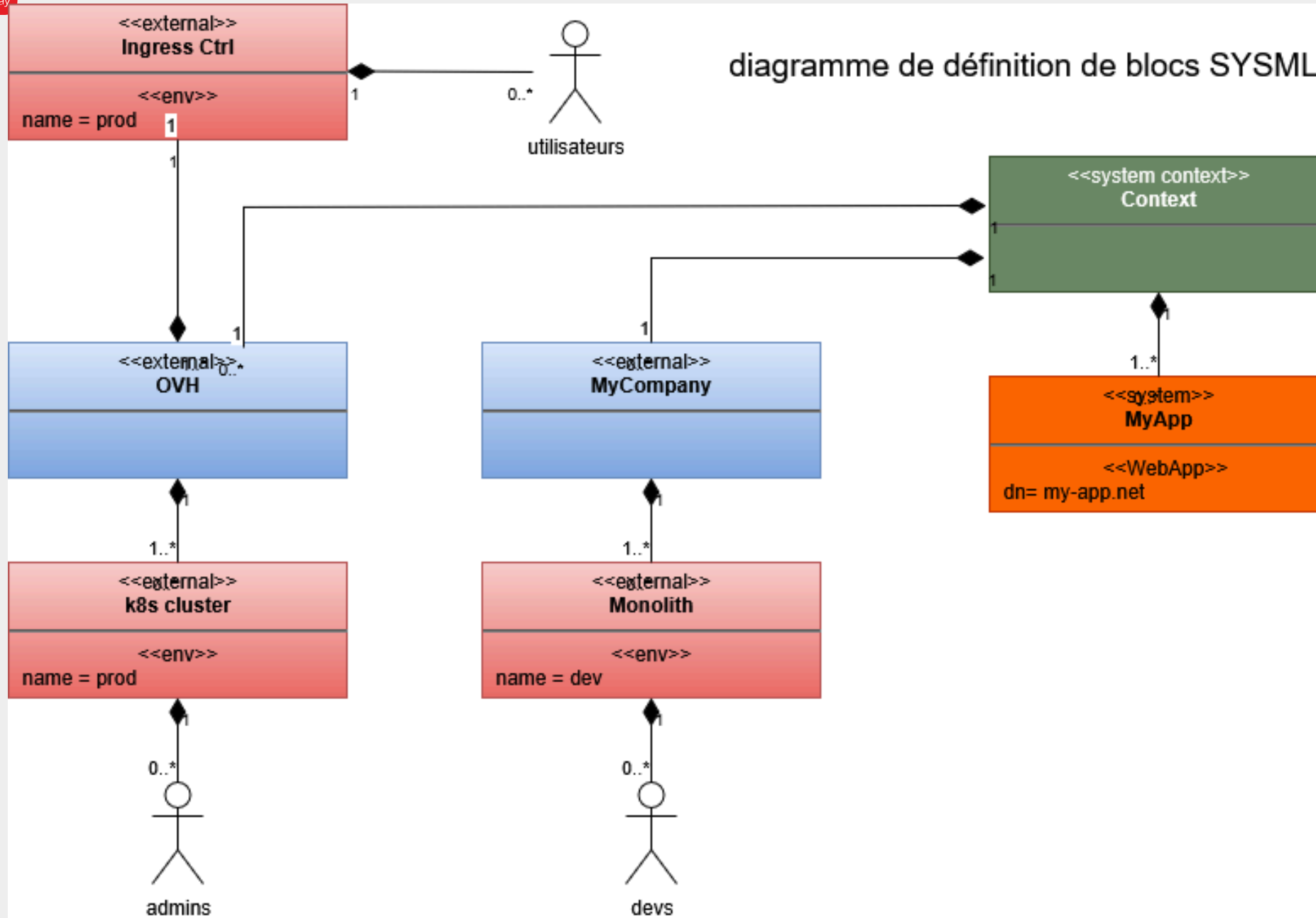
- **cartographie des contextes** pour visualiser les relations et les dépendances entre les différents bounded contexts dont un en amont *Upstream* et un en aval *Downstream*

relations remarquables:

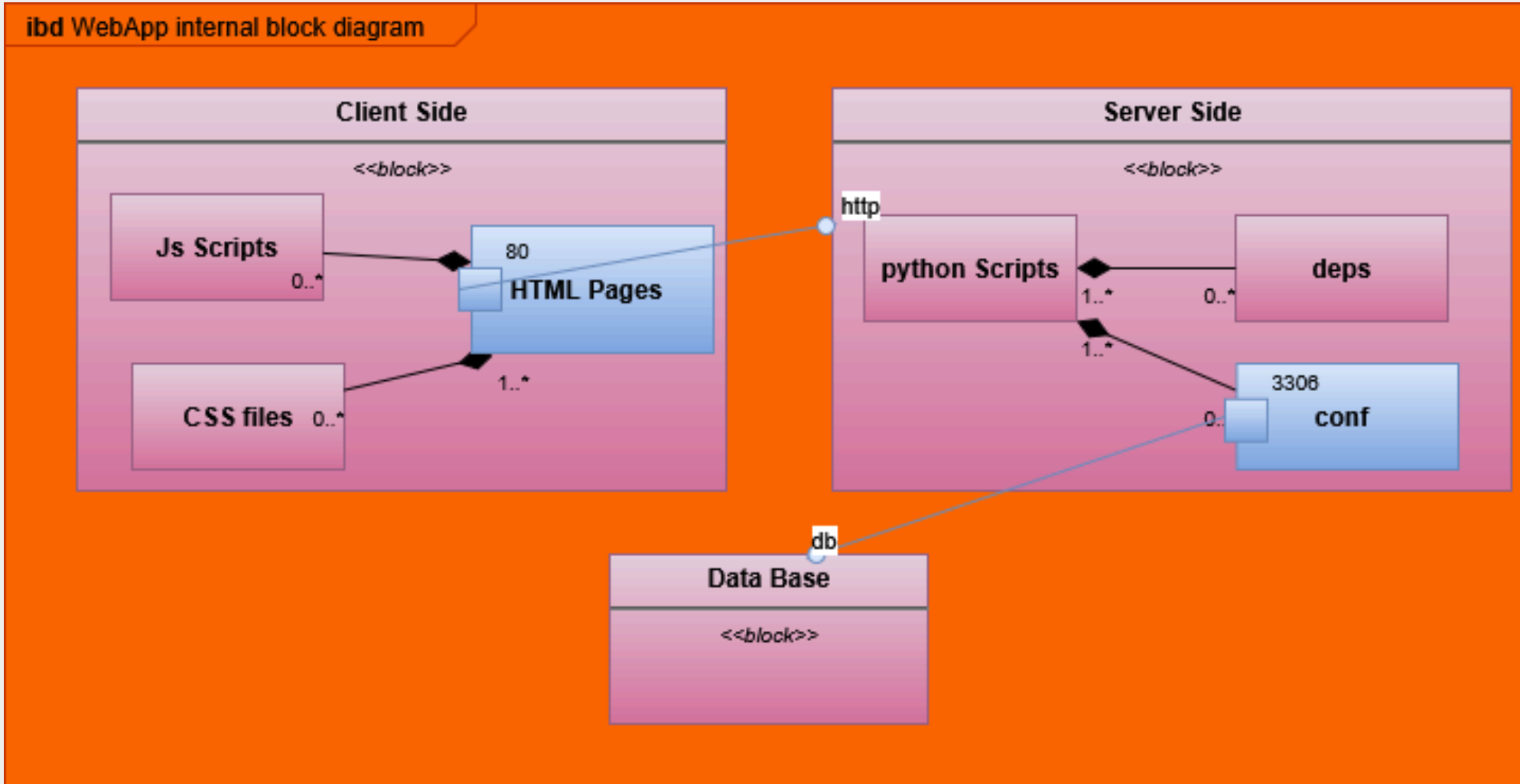
- Open Host Service (*OHS*) + Published Language (*PL*): Upstream **expose des services et son vocabulaire** aux Downstreams *REST API + OPENAPI*
- AntiCorruption Layer (*ACL*): Downstream **utilise un Adaptateur** pour traduire les services Upstream dans son vocabulaire

ANNEXES

diagramme de définition de blocs SYSML

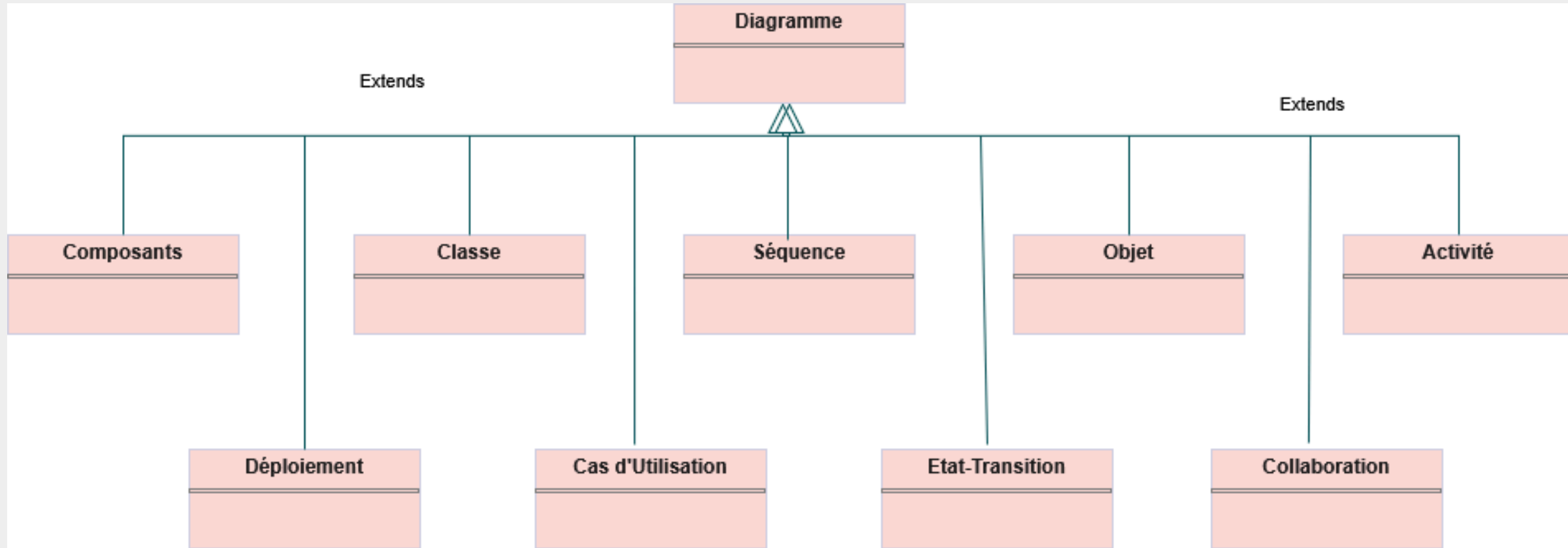


- Diagramme de block interne SYSML

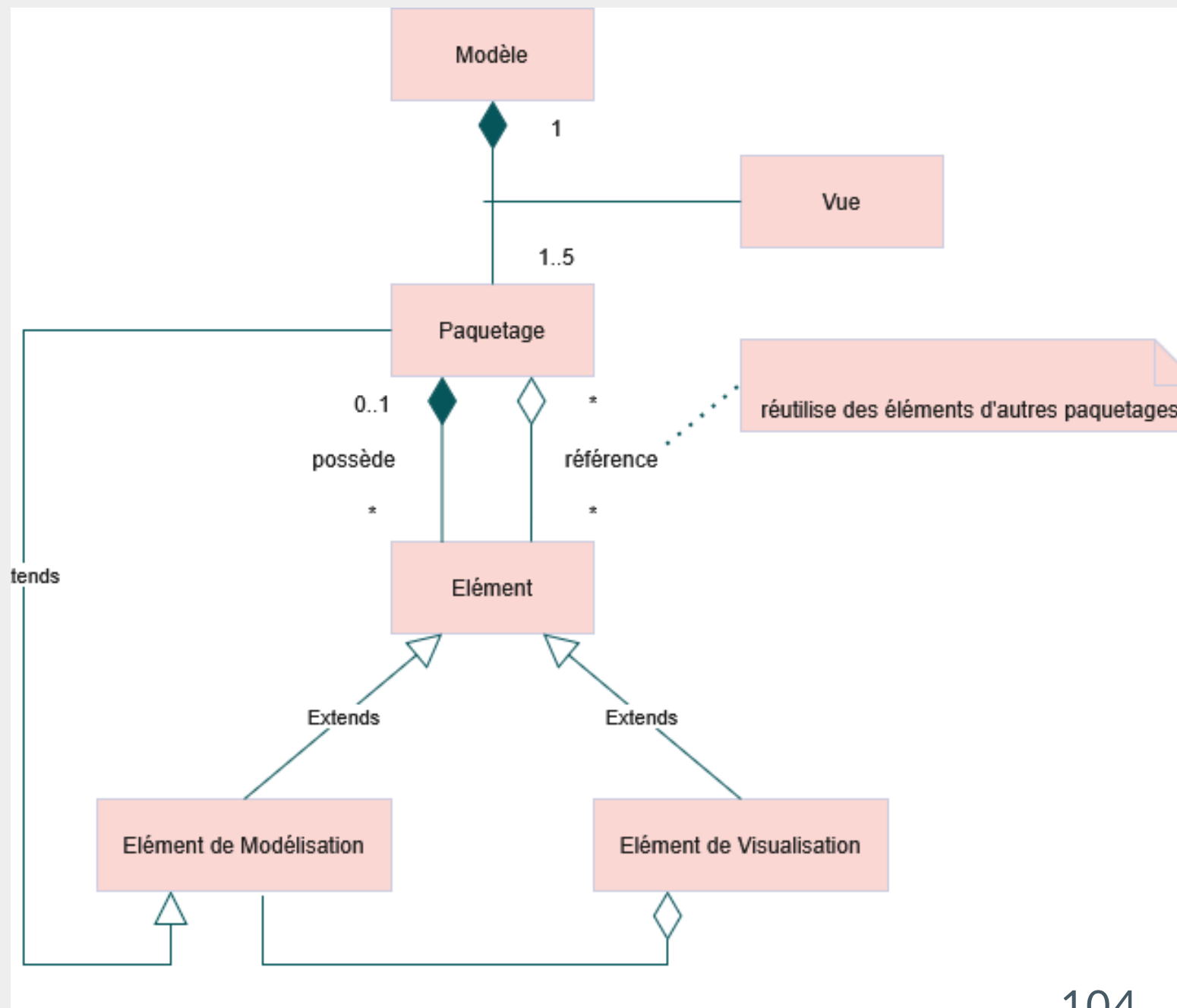


UML

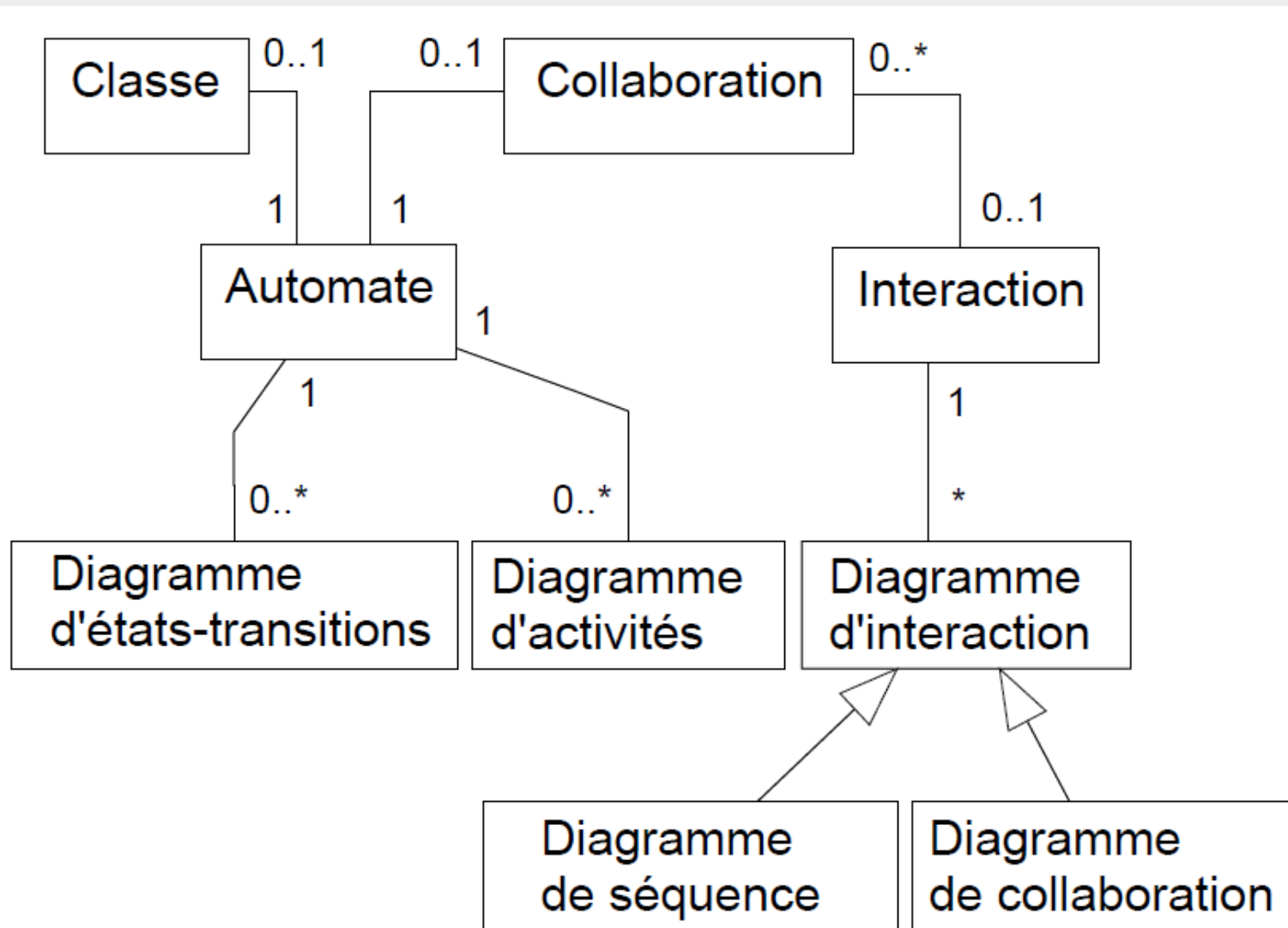
- **U**nified **M**odeling **L**anguage
- Langage de modélisation graphique standardisé
 - 14 types de diagrammes en 2.5
 - **9 diagrammes principaux**



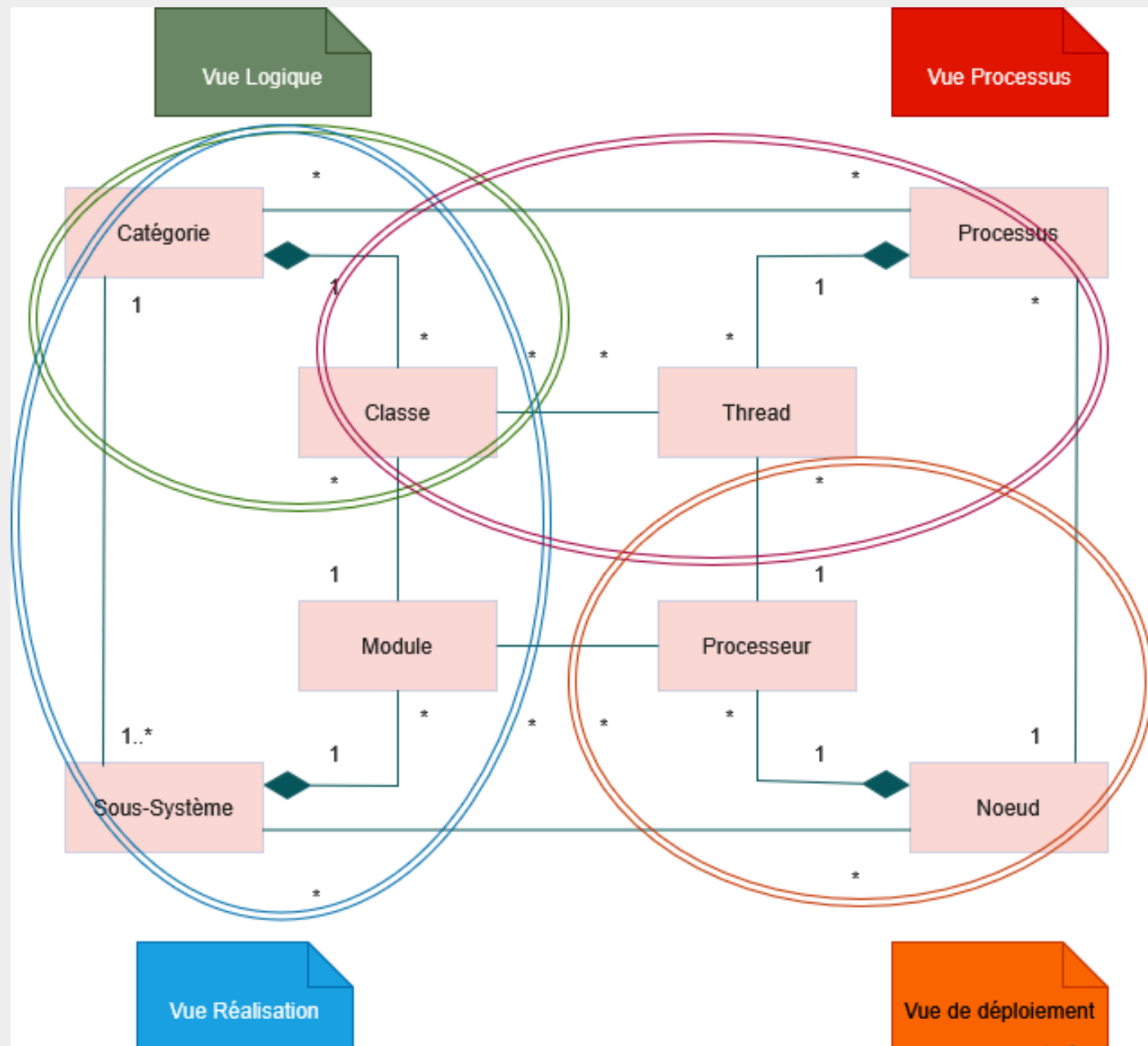
l'architecture en UML



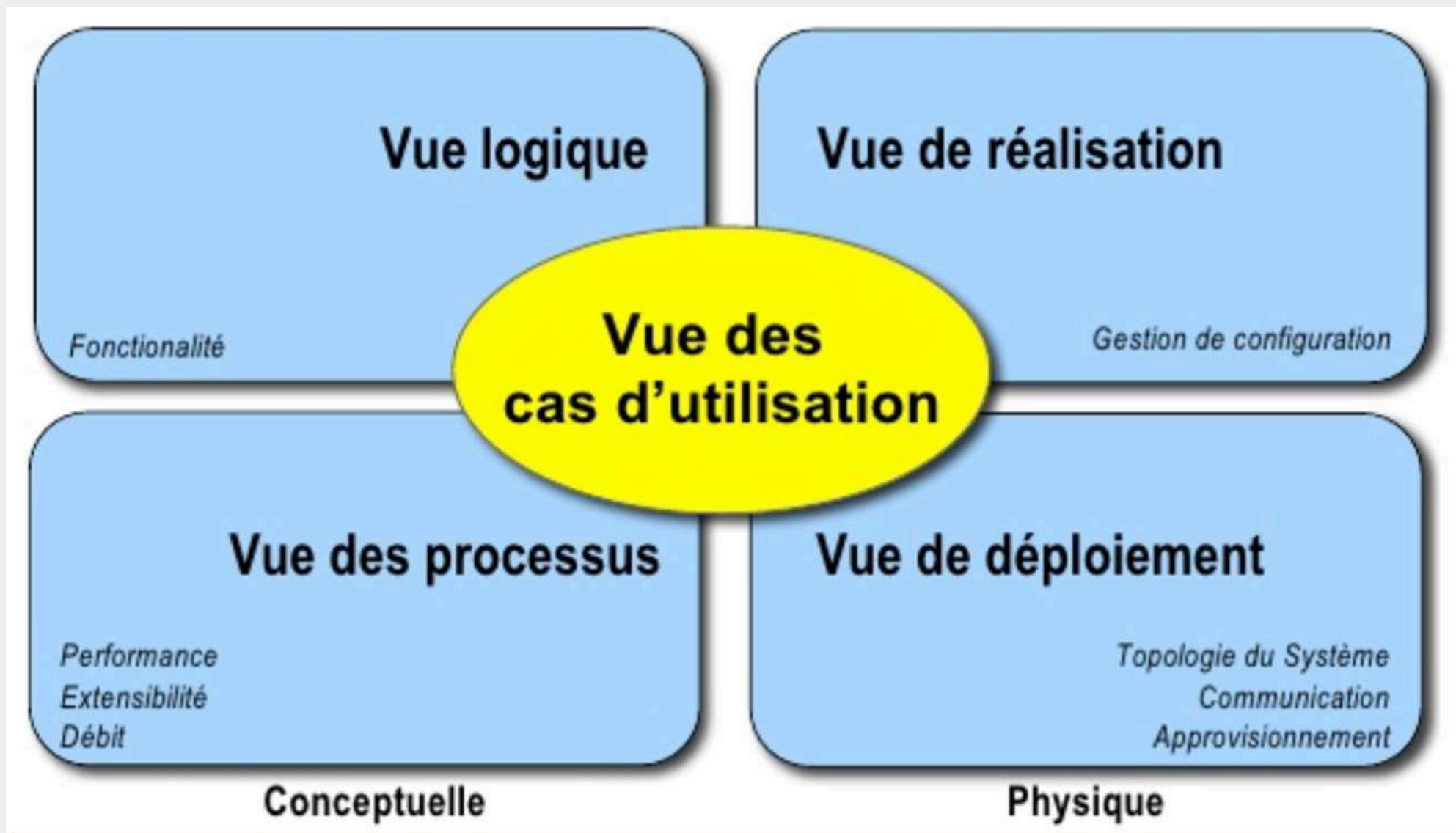
Vues "comportementales"



4 vues + 1 de Kruchten



4 vues + 1 de Kruchten



utilisation du scripting Mermaid

- générer des diagrammes d'architecture à partir d'un script
- **ici**
- injectable dans **ici** Organiser -> Insérer -> Mermaid
- convertible avec ong / pdf / svg avec **mmdc**
 - `npm install -g @mermaid-js/mermaid-cli`
 - `mmdc -i input.mmd -o output.png`

“ *utiliser un assistant IA pour convertir des sources en mermaid* ”

références sur l'architecture logicielle

- Ivar Jacobson (Use Cases),
- Grady Booch (UML),
- Robert C. Martin (Clean Code),
- Eric Evans (DDD)
- Mary Poppendieck (Lean Software Development),
- Simon Brown (C4 model)
- Gene Kim (DevOps)

bibliographie

- **TOGAF**
- **Zachman**
- **SEI**
- **série d'articles sur l'architecture explicite**
- **Matt Pocock - vocabulaire du développement assisté par l'IA**